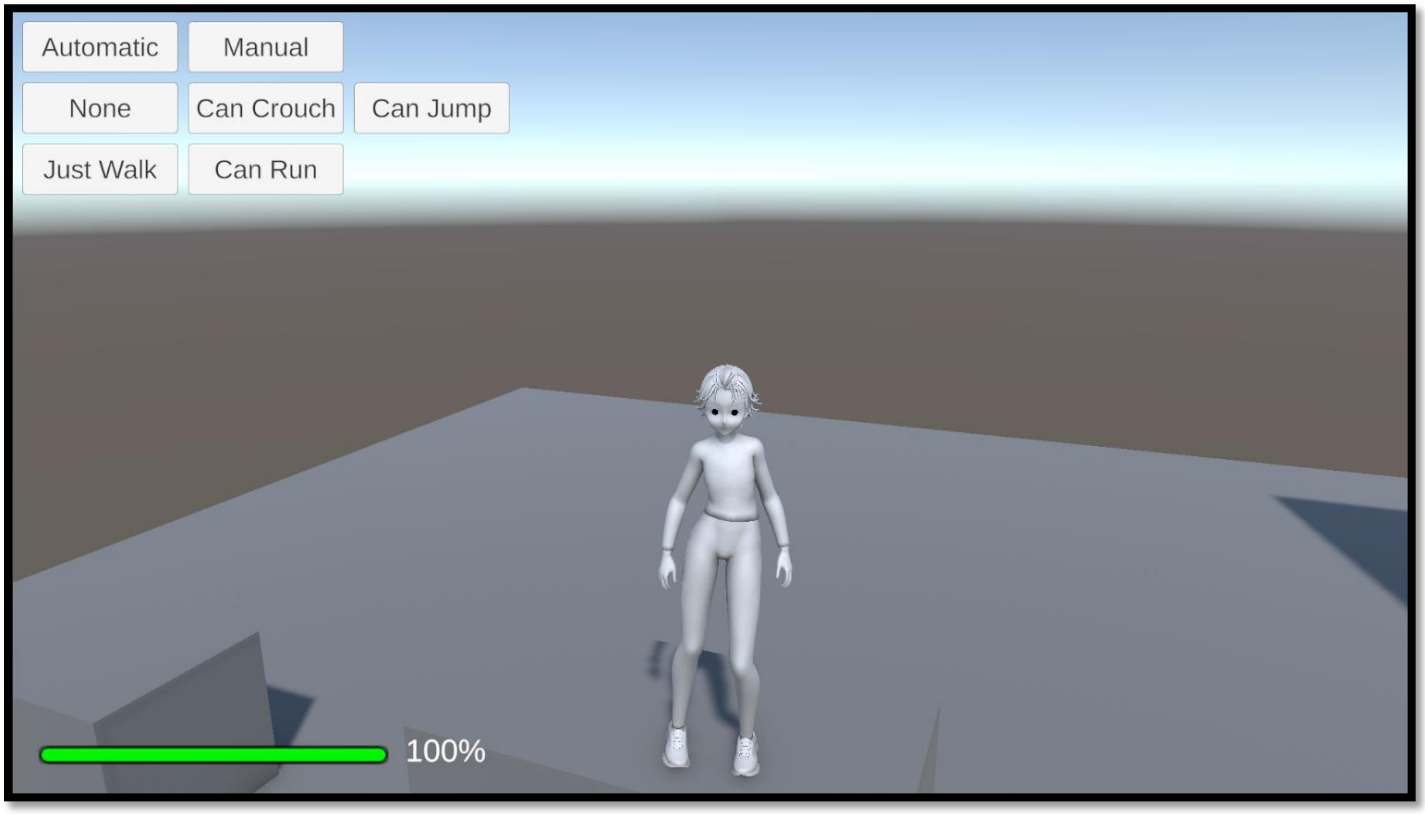

Player Controller

Documentation

_____

Creator: Lucas Gomes Cecchini

Online Name: AGAMENOM

Overview

This document will help you to use **Player Controller** for **Unity**.

With it, you can create a gameplay for your game with a modular system that's compatible with Unity's **Input System**.

It also comes with some **Predefined Controllers**.

Instructions

You can get more information from the Playlist on YouTube:

https://www.youtube.com/playlist?list=PL5hnfx09yM4LxhQjXoLlwwvmEuMW9LySa

Script Explanations

Box Collision Sensor

This script defines a **box-shaped collision detection system** in Unity that works at runtime using dynamically provided data. It allows configuration through "providers" (functions that return values), making it flexible and adaptable for different use cases. Here's a full breakdown of its functionality:

GENERAL FUNCTIONALITY

This system uses Unity's **Physics.OverlapBoxNonAlloc** method to detect if any colliders are overlapping with an invisible 3D box in the scene. It allows:

- Runtime configuration of the box (position, size, rotation).
- Filtering logic (ignore children, ignore specific tags, etc.).
- Optional logging to console.
- Optional visualization through **Gizmos**.
- Integration with an external update system (PhysicsUpdateBroadcaster), which triggers this code in Unity's FixedUpdate.

VARIABLES EXPLAINED

Detection & Filtering Controls

- enableDetectionProvider: Function returning true/false. If false, collision checking is skipped.
- enableDebugLogProvider: Function returning true/false. If true, it logs detected collisions to the Unity console.
- filterModeProvider: Returns one of the DetectionFilter options (e.g., exclude children or tags).
- referenceParentTransformProvider: Used with filtering to avoid detecting child objects of a parent.
- ignoredTagsProvider: A set of string tags to ignore when filtering detected collisions.

Box Parameters

- boxCenterProvider: Returns the world-space **center position** of the box.
- boxSizeProvider: Returns the **dimensions** of the box (width, height, depth).
- boxRotationProvider: Returns the **orientation** of the box in world space.

Physics Settings

- collisionLayerMaskProvider: Tells which Unity **layers** should be checked during collision.
- triggerInteractionProvider: Indicates whether **trigger colliders** should be considered.

Gizmo Debug Settings

- gizmoTargetObjectProvider: Specifies an object used to determine whether to draw gizmos.
- gizmoDrawingModeProvider: Determines **when** to draw gizmos (None, SelectedOnly, or Always).
- gizmosColorProvider: Defines the **color** of the box when drawn in the Unity Editor.

💥 Detection Result

- hitResults: A fixed-size array of colliders, reused every frame for performance.
- collisionDetected: A boolean flag that becomes true if a collision was found in the last check.

🧠 METHODS EXPLAINED

🏗️ BoxCollisionSensor(...) (Constructor)

- Accepts all the function providers as parameters.
- Initializes all internal variables.
- Registers CheckForCollisions to be called on every physics update (via PhysicsUpdateBroadcaster.OnFixedUpdate).
- Registers this sensor for centralized management (PhysicsUpdateBroadcaster.AddSensor).

🧹 Dispose(ref BoxCollisionSensor detector)

- Unregisters the sensor from the update system.
- Stops collision checking.
- Sets the given reference to null to avoid memory leaks or dangling listeners.

🔍 CheckForCollisions()

- Called during Unity's physics update.
- Only runs if enableDetectionProvider returns true.
- Gets box parameters (position, size, rotation).
- Performs a non-allocating OverlapBox check using Unity physics.
- Iterates through detected colliders and uses PassesDetectionFilter() to check if each collider is valid.
- If at least one valid collider is found:
    - collisionDetected becomes true.
    - Optionally logs the hit.
    - Stops checking further colliders for performance.

🧪 PassesDetectionFilter(Collider collider)

- Determines whether a collider should be accepted or ignored based on the selected DetectionFilter:
    - None: Accept all.
    - IsNotChildOf: Ignore colliders that are children of a specific transform.
    - IgnoreByTags: Ignore colliders with tags in a predefined list.

- All: Applies both child and tag filtering.

🧪 ENUMS

DetectionFilter

Used to customize post-collision filtering:

- None: No filtering.
- IsNotChildOf: Ignore children of a specific transform.
- IgnoreByTags: Ignore specific tags.
- All: Combines both filters above.

GizmoDisplayMode

Used to control editor visualization:

- None: Don't draw anything.
- Always: Always draw.
- SelectedOnly: Draw only when the object is selected.

✅ USAGE SCENARIOS

This system is ideal when you want to:

- Detect physics overlaps dynamically (not via Unity's built-in collision callbacks).
- Avoid using physics callbacks like OnTriggerEnter.
- Use a centralized physics update system (PhysicsUpdateBroadcaster) for performance.
- Filter out specific collisions (e.g., avoid self-detection).
- Draw detection boxes in the Unity Editor for debugging.

💡 EXAMPLE USE CASE

Imagine a stealth game where you want to detect if an enemy is within a certain 3D volume (e.g., a cone of vision or alarm zone), but:

- Only certain layers (e.g., Player, Allies) should be detected.
- Children of the enemy (e.g., weapon colliders) should be ignored.
- Debug visuals should be shown in the Unity Editor.
- You want all settings to be configurable at runtime (not in the Inspector).

This system allows you to do all that cleanly and efficiently.

Check Valid Angle Sensors

🎯 Purpose of the Script

This component checks if the player is **standing on a surface with an acceptable slope angle**. It does this by **shooting a ray straight down** from the player's position and measuring the **angle of the ground**. Depending on whether the slope is too steep or not, it **switches the friction material** on the player's colliders. This affects movement and sliding on slopes.

It integrates with an external system (PhysicsUpdateBroadcaster) that ensures it runs on every **physics update**, making it precise and real-time.

🧩 Breakdown of Variables

🧠 Input Providers (Functions)

These are **delegates** — functions provided from outside — that allow for dynamic configuration during runtime.

- **targetTransform**: Gives the position from which the ray should be cast (usually the player).
- **highFrictionMaterial**: Material to apply when the slope is walkable (flat or gentle slope).
- **lowFrictionMaterial**: Material to apply when the slope is too steep (causing sliding).
- **colliders**: List of the player's colliders that should have materials changed.
- **maxStableAngle**: Maximum allowed slope angle to consider the surface stable (defaults to 45°).
- **raycastDistance**: How far down to check for ground (defaults to 0.5 units).
- **debug**: Enables drawing lines and logging messages for visual debugging.

🔁 Internal Variables

- **isValidAngle**: Boolean flag that tells whether the last surface detected was walkable or too steep.
- **raycastHits**: A reusable array used to store ground hits from the raycast, reducing memory allocations.

🛠️ How the System Works

1. Initialization

When this component is created:

- It stores all the function delegates for later use.
- It sets default values if any delegate was not provided.
- It registers the method CheckRaycast to run during every physics update (via PhysicsUpdateBroadcaster).

2. Collision Detection (CheckRaycast)

This is the core logic:

🔽 Step 1: Cast a Ray Downward

- The system sends a **ray** slightly below the player's position, pointing straight downward.
- The length of the ray is configurable (default is 0.5 units).
- It checks what it hits using an efficient method that **avoids memory allocations**.

📦 Step 2: Evaluate Ray Hits

- If **no surface is hit**, the system sets isValidAngle to false and logs that there's no ground.
- If **a surface is hit**, it sorts the hits by proximity (closest first) and checks each one.

🔎 Step 3: Filter Invalid Hits

Each hit is inspected:

- It skips colliders that are part of the player itself.
- It skips hits where the surface is too vertical (almost a wall).

📐 Step 4: Measure Slope Angle

- The angle between the hit surface and the upward direction is calculated.
- If the angle is **less than or equal to the max stable angle**, it's considered **walkable**.

🎨 Step 5: Apply Material

- Based on the slope, it chooses the **high or low friction material**.
- It updates all player colliders with the chosen material to control sliding or sticking to the ground.

✅ Step 6: Final Result

- It sets the isValidAngle flag to true or false depending on the slope.
- Only the **first valid ground hit** is processed; the loop exits after that.

3. Disposal (Dispose)

- When the object is no longer needed, this method unregisters CheckRaycast from the update system and clears its reference.

🧠 Custom Sorter

RaycastHitComparer

- This internal class helps sort raycast results so that the closest hit is processed first.
- This ensures accuracy when multiple surfaces are below the player.

🧪 Usage Example (Conceptual)

Imagine you're creating a character controller where the player:

- Can walk on ground surfaces.
- Should **slide off steep slopes**.
- Needs **stable footing to jump or climb**.

You would use this system to:

- Detect whether the ground is walkable based on slope.
- Switch friction to increase or reduce grip.
- Prevent jumping when not grounded or on a steep slope.
- Show debug lines in the editor for development clarity.

🟢 Visual Debugging

When enabled:

- It draws **green lines** on walkable surfaces.
- It draws **yellow lines** on steep surfaces.
- It logs helpful messages about slope angles and materials.

✅ Summary

Feature	Description
Ground Detection	Uses raycast below player.
Slope Angle Evaluation	Compares surface angle with threshold.
Friction Material Switching	Applies high or low friction material dynamically.
Filtering Logic	Ignores player's own colliders and steep vertical surfaces.
Debug Support	Draws rays and logs slope info if enabled.
Optimization	Avoids runtime allocations and unnecessary computations.
Update Integration	Runs reliably on every physics step via external broadcaster.

Custom Player Data

🎯 Purpose of the Script

This class provides a way to **save and load custom player data** as a **JSON string**. It supports both **standard types** (numbers, strings, booleans), **Unity types** (such as vectors and rotations), and **enums**, and it preserves type information during serialization.

This means you can:

- **Build a dictionary** with various player data (like position, health, score).
- **Serialize it to a JSON string** to save it.
- **Deserialize it back into a dictionary** later, reconstructing all original values.

📦 Main Components

1. Method: SaveData

This method converts a dictionary of key–value pairs into a JSON string.

✅ Parameters:

- dataBuilder: A function that returns a dictionary with the data to be saved. Each entry in the dictionary is a key (string) and a value (object).

🔄 How it works:

1. It gets the dictionary from the function.
2. It prepares a list to hold all key–value pairs in a structured way (EntryList).
3. For each item:
    - If the value is null, it skips it.
    - If the value is an **enum**, it stores its name as a string.
    - If it's a **Unity type** (e.g., Vector3, Quaternion), it uses Unity's built-in JSON serializer.
    - Otherwise, it converts the value to a string.
4. It also saves the **type information** (so it can know how to interpret the value when loading).
5. At the end, it serializes the entire list of entries to a JSON string and returns it.

2. Method: LoadData

This method takes a JSON string and turns it back into a dictionary with proper types.

✅ Parameters:

- json: A string containing previously saved player data.
- applyData: A function that will receive the reconstructed dictionary.

🔄 How it works:

1. It reads the JSON into an EntryList (list of key–value–type entries).
2. It creates a new dictionary to hold the recovered data.
3. It uses a predefined set of **type converters** for common types:
    - Numbers (int, float), bool, string
    - Unity types (Vector2, Vector3, Vector4, Quaternion)
4. For each entry:
    - It checks the type metadata.
    - If it's an **enum**, it converts the string back to the enum.
    - If it matches one of the known types, it uses the correct converter.
    - If not, it tries a general conversion.
    - If conversion fails, it logs a warning and skips the entry.

5. When finished, it passes the dictionary to the function provided via applyData.

📦 Supporting Classes

📄 Entry

Represents one saved item. Each entry includes:

- A key (the name of the data).
- The value (as a string).
- The full type name of the value (needed for correct deserialization).

📂 EntryList

A container that holds multiple Entry objects.

- This is needed because Unity's JSON utility can't directly serialize dictionaries.
- So instead, a list of structured entries is serialized instead.

🧪 Usage Concept

Imagine you want to **save and load the state** of a player. You could do this:

- On save:
  - Build a dictionary:
    - "position" → player's position (Vector3)
    - "health" → current health (float)
    - "isAlive" → boolean flag
    - "movementMode" → enum indicating walking, running, etc.
  - Call SaveData() with a function that returns this dictionary.
  - Store the returned JSON in a file or PlayerPrefs.
- On load:
  - Read the JSON from where you stored it.
  - Call LoadData(), passing the JSON string and a function that takes the recovered dictionary and applies it back to your player.

🧠 Summary Table

Feature	Description
Serialization	Converts a dictionary to a JSON string with type metadata.
Deserialization	Reconstructs the dictionary from a JSON string.
Supports Unity types	Handles Vector2, Vector3, Vector4, Quaternion using Unity's JSON utility.
Supports enums	Saves enums as strings and re-parses them correctly.
Type safety	Each entry stores its type to ensure proper reconstruction.
Extensible	More types can be supported by adding new converters.

Lightweight	Uses Unity's built-in JSON serializer; no third-party tools needed.

💡 When to Use

- Saving and loading player progress.
- Transferring complex player states between scenes.
- Creating debug snapshots of runtime data.
- Handling custom save systems that need to persist data across sessions.

Mono Behaviour Extensions

🎯 Purpose of the Script

This script allows developers to **dynamically access properties** (like IsGrounded, IsRunning, etc.) from **any player controller script**, even if the script is not directly connected by inheritance. It uses **reflection** to **read property values by name at runtime**, and provides a **unified interface** (IPlayerMovementState) so different systems can interact with player movement state consistently.

This is particularly useful for tools, sensors, or logic that must work with **various player controller scripts** without knowing their exact class structures.

📦 Main Components

1. 🔍 Extension Method: TryGetProperty

This is a **utility function** that allows any behavior script (like a movement controller) to **attempt reading a property by name** and type.

✅ What it does:

- Takes the name of a property (as text).
- Uses reflection to search for a matching **public** property on the object.
- If it exists **and** has the expected data type, it reads and returns its value.
- Otherwise, it returns a default (usually false or null) and indicates failure.

🧪 Example Use Case (conceptual):

Suppose you have a character controller that has a public property called IsGrounded. You can ask the object:

"Hey, do you have a property named 'IsGrounded' of type boolean? If yes, give me the value."

This allows flexibility to work with **any kind of player controller**, as long as it exposes properties with expected names.

2. 🧩 Interface: IPlayerMovementState

This defines a **contract**: a consistent set of properties that describe a character's movement state.

👇 It includes four boolean properties:

- IsGrounded — Is the character touching the ground?
- IsMoving — Is the character moving?
- IsCrouching — Is the character crouching?
- IsRunning — Is the character running?

Any class that implements this interface promises to **provide those four answers**.

3. 🧠 Adapter: PlayerMovementStateAdapter

This is a **wrapper class** that acts as a bridge between **unknown controller scripts** and **code that expects the IPlayerMovementState interface**.

✅ What it does:

- You give it any behavior script (like a third-party controller or a custom one).
- It tries to read the movement-related properties (IsGrounded, IsMoving, etc.) from that script **using the reflection helper**.
- If the target object has those properties, it reads and returns their values.
- If they're missing or have the wrong type, it defaults to false.

🤖 How it works internally:

Each property in this adapter calls the TryGetProperty method. This makes it **safe and dynamic**, without throwing errors if the target doesn't match exactly.

🛠️ Summary of Variables and Methods

Name	Type	Purpose
TryGetProperty	Method	Dynamically reads a named property from a behavior if it exists and matches the expected type.
IPlayerMovementState	Interface	Describes what a movement state should include (4 booleans).
PlayerMovementStateAdapter	Class	Allows any behavior script to pretend it implements the movement state interface by using reflection.
target	Variable	The actual behavior instance from which the adapter tries to extract data.

🧪 When to Use This System

- When you are developing a **generic system** (like AI, sensors, or physics helpers) that should **work with multiple character controllers**.
- When you **don't want to directly reference or depend** on specific controller classes.
- When your system only needs to **read standard movement states**, like walking, crouching, or grounded status.
- When integrating with **external controllers or third-party assets** that cannot be modified but still expose readable properties.

🔄 Benefits of This Design

- 🔌 **Plug-and-play**: Works with any object exposing correctly-named public properties.
- 🧩 **Decoupled**: No need for hard dependencies between your systems and the player controller.
- ⚡ **Safe**: Uses defensive programming to avoid errors when properties are missing.
- 🔄 **Reusable**: Can be expanded to include more properties (e.g., jumping, sliding, swimming).

✅ Final Summary

Component	Role
Reflection Method	Reads values by property name and type safely.
Interface	Defines what a movement-aware object must expose.
Adapter	Connects any compatible script to systems expecting standardized movement data.
Usage	Ideal for modular systems like animation, audio, sensors, or AI that need to check player state generically.

On Input System Event

🎮 Purpose of This Script

This script is a **generic event dispatcher** that listens to **input actions** (like pressing a button or moving a joystick) and triggers specific functions when the input is:

- **Pressed** (just activated),
- **Held** (continuously active),
- **Released** (deactivated).

It works with **any type of input value** such as booleans, floats, or 2D vectors (like those from analog sticks or triggers). The system is built on top of Unity's **Input System** and is **generic and reusable**.

📦 Core Components

🔁 1. Input State Dictionary

There is a global list (a dictionary) that keeps track of all the input actions the system is monitoring. Each action has a corresponding **state object** that stores:

- The last known input value.
- Whether the input is currently active or held.
- The functions to run when the input is pressed, held, or released.
- A condition to enable or disable evaluation at runtime.

🧠 2. Internal "State" Class

This internal class holds everything related to one specific input action.

Variables:

- **action**: The actual input being tracked.
- **lastValue**: Stores the last read input value.
- **wasHeld**: A flag indicating whether the input was active during the previous check.
- **OnPressed**: Function(s) to call when the input changes from inactive to active.
- **OnHold**: Function(s) to call repeatedly while input stays active.
- **OnReleased**: Function(s) to call when input changes from active to inactive.
- **Enabled**: A function that determines if this action should be processed.

Main Function:

The state object includes a method that runs every frame and checks the current input value:

- If the input is now active and wasn't before, it triggers the **Pressed** event.
- If it stays active, it keeps calling the **Hold** event.
- If it becomes inactive, it triggers the **Released** event.
- It updates its internal state to reflect the current condition.

🧪 3. Input Evaluation Logic

To determine whether an input is "active" or not, it checks if the value is not zero. Here's how it works for different types:

- A small float (e.g., 0.001) is considered inactive.
- A vector with very low magnitude is considered inactive.
- An exact zero (for numbers or vectors) is inactive.
- A true boolean is active; false is inactive.
- A default quaternion is considered inactive.

- Any custom type is compared to its default value.

⚙️ Registration Methods

The script includes multiple ways to **register and unregister** functions for a given action:

Function	What It Does
RegisterPressed	Adds a function that runs **once when the input is first pressed**.
RegisterHold	Adds a function that runs **continuously while the input is active**.
RegisterReleased	Adds a function that runs **once when the input is released**.
UnregisterPressed/Hold/Released	Removes previously added functions.
Clear	Removes all registered functions for a specific input.

These methods let you **fully control what happens** in response to different input events.

🧩 Fluent Setup with "WithAction"

To make things easier, the script allows a **fluent-style configuration**:

1. You give it an **Input Action Asset** (like your input map).
2. You specify the **name or path of the input** (for example: "Gameplay/Jump").
3. Optionally, you give a function that controls **whether the input should be evaluated** (for example: "only if the menu is not open").

Then you can chain method calls like:

- .OnPressed(...)
- .OnHold(...)
- .OnReleased(...)
- .UnbindAll() to remove all callbacks.

This makes the setup clean and readable.

🐵 OnInputSystemEventConfig

This class handles the fluent configuration described above.

It simplifies the setup process and helps you bind everything in a compact and readable way.

- When initialized, it locates the specific input inside the asset.
- It enables that input.
- It allows you to attach callbacks with chained methods.

📦 Summary of Key Elements

Name	Type	Role
states	Dictionary	Tracks all monitored inputs and their internal states.
State	Class	Holds previous value, flags, and registered events for one input.
Update	Method	Called every frame to check for transitions and trigger events.
IsNonZero	Method	Determines if a value is considered "active".
Register/Unregister	Methods	Add or remove event callbacks.
Clear	Method	Remove all events for an input.
WithAction	Method	Starts fluent configuration.
OnInputSystemEventConfig	Class	Fluent wrapper for input setup.

🧠 When and Why to Use This Script

This system is ideal when:

- You want to track player inputs **without writing tons of repetitive code**.
- You need to handle complex input logic like **press** → **hold** → **release**, especially for analog inputs.
- You want to **rebind input actions easily** using action paths.
- You want your input logic to be **modular, clean, and reusable**.

It works seamlessly across **any input type** — from keyboard keys to gamepad sticks to touch.

🎮 Example: Handling Player Jump Using This System

Let's say you have a **player character** and you want to:

- **Start a jump** when the player presses the "Jump" button.
- **Charge a jump** if the button is held.
- **Cancel or land** when the button is released.

You'll use this dispatcher to listen to the input like this:

🧩 Step 1: Reference the Input Asset

Suppose your InputActionAsset has an action called:

"Gameplay/Jump"

This is the path you'll use.

⚙️ Step 2: Setup in Your Script

Imagine you're inside a Start() or OnEnable() function of your PlayerController. Here's what you'd *say in code*, and I'll explain it below.

💡 Pseudocode (translated to plain English)

```
Start a new listener for the input called "Gameplay/Jump".

While the input is being held (button is down):
    Continuously add power to the jump (e.g., charge up).

When the input is first pressed:
    Begin the jump sequence (e.g., play animation, start timer).

When the input is released:
    Launch the jump using the charged power.
```

✅ Real Usage Summary

Here's what the code would mean in plain English:

```
OnInputSystemEvent<float>
    .WithAction(playerInput, "Gameplay/Jump", () => !isPaused)
    .OnPressed(value => BeginJump())
    .OnHold(value => ChargeJump(value))
    .OnReleased(() => LaunchJump());
```

Explanation:

- playerInput: This is your InputActionAsset containing all your input bindings.
- "Gameplay/Jump": The path to the action inside the asset.

- () => !isPaused: Optional logic — this means "only process this if the game is not paused".
- OnPressed: Runs when the jump button is **first pressed**.
- OnHold: Runs **every frame** while the jump button is **held down**.
- OnReleased: Runs **once** when the player **releases** the button.

🧠 What Happens at Runtime

Player Action	What the Script Does
Presses jump	Triggers BeginJump() (maybe shows charging animation)
Holds jump	Continuously calls ChargeJump(value) with current analog strength
Releases jump	Triggers LaunchJump() to perform the final jump

🧪 Bonus: Another Example with Movement

Let's say you want to listen to **movement input** (Vector2) to trigger movement events.

```
OnInputSystemEvent<Vector2>
    .WithAction(playerInput, "Gameplay/Move")
    .OnPressed(dir => Debug.Log($"Movement started in direction {dir}"))
    .OnHold(dir => MoveCharacter(dir))
    .OnReleased(() => Debug.Log("Movement stopped"));
```

This would track movement from a joystick or WASD, and log or handle actions accordingly.

✅ Summary

With just a few lines, this system lets you:

- Set up **multiple types of input values** (float, Vector2, bool, etc.).
- React to **pressed**, **held**, or **released** events with **clean and modular callbacks**.
- **Avoid boilerplate** like manual polling or input condition checks.

Player Jump Controller

🎯 Purpose of the Script

This script handles **jumping mechanics** for a character controlled with a **Rigidbody**. It includes:

- **Multi-jump** support (like double/triple jump),
- **Coyote time** (a short window to jump after falling off a platform),
- **Cooldown prevention** (to avoid detecting false ground collisions right after a jump).

🧱 What Each Variable Does

Variable Name	What It Is	What It Does
targetRigidbody	A function that gives access to the character's physics body.	This is where the jump force will be applied.
isGrounded	A function that returns whether the player is touching the ground.	Used to decide when to reset jumps and when coyote time starts.
maxJumps	A function that gives the maximum number of jumps allowed.	Example: 2 = double jump.
coyoteTime	A function that tells how long after falling the player can still jump.	Adds forgiveness to timing.
jumpCount	Internal counter of how many jumps have been used.	It resets when the player lands.
lastGroundedTime	The last time (in seconds) the player was on the ground.	Used to measure coyote time.
wasGroundedLastFrame	Whether the player was grounded in the previous physics step.	Used to detect transitions like landing or falling.
jumpCooldownTime	The time until which "grounded" checks are ignored.	Prevents false grounded detection during takeoff.
groundedIgnoreDurationAfterJump	A constant that defines how long to ignore grounded status after jumping.	Default is 0.1 seconds.
coyoteJumpUsed	Indicates whether the player already used the coyote jump.	Prevents abusing the coyote feature.
isInCoyoteWindow	Tells whether the current time is still within the coyote window.	Enables coyote jump logic.

🔁 How It Works Step-by-Step

1. Initialization

When the system starts, it receives **functions** that return values:

- Rigidbody to apply forces to,
- Grounded state,
- Max jumps allowed,
- How long coyote time should last.

These functions allow the controller to be reused with any character.

2. Update Loop (called every physics tick)

This is where the controller evaluates the state of the character.

What it checks and updates:

- Whether the player is **currently grounded**.
- Whether to **ignore grounded** (for a short time after jumping).
- If the player **landed**, it resets the jump counter and allows jumping again.
- If the player **left the ground**, it starts a **coyote timer**.
- It also determines whether we're still in the **coyote window** based on time.

This loop ensures that every frame, the controller knows:

- How many jumps are left,
- Whether the player is eligible for a coyote jump,
- Whether grounded detection should be ignored.

3. OnJump()

This method is **called when the player presses the jump button**.

It does several checks before jumping:

- Has the player landed?
- Are we within coyote time?
- Does the player have extra jumps left?

If yes, the controller:

- Resets vertical velocity to make the jump consistent,
- Applies upward force using impulse,
- Marks the cooldown period to **ignore grounded**,
- Updates jump count and flags if a **coyote jump** was used.

It then returns true to confirm the jump happened, or false if not allowed.

🔄 How to Use It (Plain English)

1. Create a PlayerJumpController, giving it:
    - The Rigidbody of the player,
    - A function that tells if the player is on the ground,
    - Optional: how many jumps you want (default is 1),
    - Optional: how long coyote time lasts (default is 0).
2. In your game code, when the player presses jump:
    - Call OnJump() with a desired jump force (e.g. 5).
3. Inside physics updates, the controller tracks:
    - Grounded transitions,
    - Jump cooldowns,
    - Coyote time window,
    - How many jumps are used.
4. When the player lands again, everything resets automatically.

🧠 Example Use Case

Imagine a 2D or 3D platformer where:

- The player can double jump.
- You allow jumps 0.2 seconds after walking off a cliff.
- You want to avoid "floor" detection mid-jump due to ceilings or stairs.

This controller does all that automatically — no need to manually track jump states or collisions.

Player Move Controller

🎯 Purpose of the Script

This script is designed to control the **movement of a Rigidbody-based player character**. It moves the character using **horizontal inputs (X and Z axes)** and allows for:

- Custom movement speed,
- Optional logic to block movement when an obstacle is detected (e.g., a wall).

🔩 What Each Variable Does

Name	Description
targetRigidbody	A function that provides the physics object (Rigidbody) to control. It's the core object that will be moved.
speed	A function that returns how fast the player should move. If no custom speed is provided, it defaults to 3.

These variables are stored as **functions** (called delegates) so that their values can be dynamic — they can change during gameplay.

🧠 Constructor: Setting Up the Controller

When creating a new movement controller, you give it:

- The Rigidbody to control,
- Optionally, a way to define how fast the player moves,
- Optionally, a rule to detect when movement should be blocked (like hitting a wall).

If you don't provide values for speed or obstacles, it uses default values.

🔁 Main Methods and What They Do

🟢 OnMove(directionXZ)

This is the method you call when the player **should move** (usually during input). It expects a **2D direction** (with X and Z components).

What happens inside:

1. It checks whether movement should be **blocked**. If yes, it calls OnStop() to halt the player.
2. If not blocked:
    - It gets the Rigidbody.
    - It gets the movement speed.
    - It calculates the movement vector based on input and speed.
    - It **applies velocity** to the X and Z axes only, while **preserving Y velocity** (so it doesn't interfere with jumping or falling).

So, this keeps gravity and vertical motion untouched, only controlling horizontal movement.

🔴 OnStop()

This method **stops horizontal movement**, usually called when:

- The player hits a wall or obstacle,
- The movement input ends,
- Or when the game logic wants to freeze the player's motion temporarily.

It sets the velocity in the X and Z axes to 0, while **keeping the current Y velocity** (e.g., if the player is still falling or jumping, that motion continues).

🧹 Dispose(ref controller)

This is a helper method used to **safely remove** or reset the controller instance. It simply sets the reference to null, effectively discarding the controller.

⚙️ How You Would Use This in a Game

1. **Create the controller** by passing in the player's Rigidbody, a function to return the movement speed (optional), and a function to check for obstacles (optional).
2. **Each time the player provides movement input**, call OnMove() and give it a direction vector (e.g., from a joystick or keyboard).
3. If movement should stop — either due to no input or hitting something — call OnStop().

✅ Why This Is Useful

- It **keeps gravity and jumping separate** from horizontal movement, making the system modular.
- You can change movement speed dynamically (e.g., sprinting).
- You can block movement during cutscenes, attacks, or when the character is stunned.
- It's **simple, reusable**, and works in both 2D and 3D (as long as movement is on the XZ plane).

Player Rotation Controller

🎯 Purpose of the Script

This script smoothly rotates a game object based on directional movement input (in the **XZ plane**, usually horizontal movement in 3D). It uses interpolation to make the rotation fluid rather than instant. The logic runs on every **fixed update frame**, which is ideal for physics-related actions like rotation.

🧩 What Each Variable Does

Variable	Purpose
targetTransform	A function that returns the object's **Transform**, which is the thing to be rotated.
direction	A function that returns the **2D movement direction** (on X and Z axes). For example, this could be input from a joystick or keyboard.
speed	A function that returns how **fast the object should rotate**. If none is provided, a default value of 10 is used.
magnitude	A function that defines the **minimum strength of movement** required to trigger rotation. This helps avoid small jitters or unwanted rotation when input is near zero. Defaults to 0.01.

🔧 How the Script Works Internally

🛠️ Constructor – When You Create the Controller

When the controller is initialized:

- It stores the references to the movement direction and the transform that will rotate.
- It assigns default values for rotation speed and sensitivity threshold if they're not given.
- It **registers itself to a fixed-update broadcaster**, meaning it will update rotation every physics frame.

🔄 UpdateRotation – Called Automatically on Every Fixed Frame

This is the core of the script. It runs continuously and handles the rotation logic.

Steps:

1. **Gets the movement direction** as a 2D vector (like from a joystick).
2. If the direction's strength is above a minimum threshold (defined by magnitude), it continues.
3. Converts the 2D direction into a 3D direction (X stays X, Y becomes Z, and Y axis is set to 0 to ignore vertical rotation).
4. Calculates the **target rotation** based on the movement direction.
5. **Smoothly rotates** the object from its current orientation toward the new direction using interpolation (Lerp), applying the specified speed.

This makes the rotation feel natural and fluid rather than snappy or rigid.

🧹 Dispose – Cleaning Up the Controller

This method stops the controller by:

- **Removing its fixed-update behavior** so it no longer rotates the object.
- **Nullifying the reference**, ensuring it won't be used or updated again.

This is useful for disabling or destroying the controller safely during gameplay.

👾 Typical Use Case in a Game

You would use this in a character controller or AI to make sure the object **faces the direction it is moving**. For example:

- A player holding W + D (diagonal) would make the character rotate smoothly to face forward-right.
- If movement stops, no rotation occurs because of the minimum magnitude check.

- Rotation speed can be dynamically adjusted (e.g., fast when sprinting, slow when sneaking).

✅ Why It's Useful

- Keeps character facing direction in sync with movement.
- Smooth rotation improves the player's visual experience.
- Fully modular: you can use this on **any object**, not just players.
- Runtime customization: you can plug in different speed values or movement sources at any time.

Player Speed Control

🔍 What the Script Does

This script manages how a player moves between **walking** and **running**, using a **stamina system**. It:

- Controls the **current movement speed** depending on the player's state.
- Handles **stamina depletion** while running.
- Handles **stamina regeneration** when not running.
- Disables running when stamina runs out and restores it when enough stamina has regenerated.

It works on each **physics frame** (a fixed update cycle).

🧠 Conceptual Overview

Imagine your player has two speeds:

- **Normal speed** (walk)
- **Fast speed** (run)

Running consumes stamina over time. When stamina reaches zero, the player is forced to walk. If they stop running or moving, stamina regenerates. Once it's high enough again, the player is allowed to run.

🔢 Variable Breakdown

📌 Configuration Functions (Passed by External Systems)

These are functions that return values dynamically (e.g. from character stats or settings):

Name	What It Represents
normalSpeed	The walk speed.

fastSpeed	The run speed.
maxStamina	The maximum amount of stamina the player can have.
minimumAmountVigor	The minimum stamina required to allow running again.
staminaDepletionRate	How fast stamina drains while running.
staminaRecoveryRate	How fast stamina regenerates while not running.
isMove	A function that returns whether the player is moving.

📦 Internal State Variables

Name	What It Does
useStamina	True if the stamina system is active.
currentStamina	The player's current stamina level.
subscribedToUpdate	Tracks whether the script is registered to update each physics frame.
isFast	True if the player is trying to run.
haveEnoughStamina	True if stamina is above the required minimum to run.

🧪 Public Values You Can Read From

These can be used by other systems, like UI or animations:

Name	What It Shows
resultingVelocity	The player's current movement speed (normal or fast).
hasStamina	True if the player is running and has enough stamina.
staminaPercentage	A percentage (0–100) of how much stamina remains.

🧭 Method Explanations

📦 Constructors

There are **two ways to create the controller**:

1. **Without stamina system:** Only needs walk and run speed.
2. **With stamina system:** Also needs stamina-related parameters and a movement check.

If the stamina version is used, it **automatically registers to the fixed update loop** so stamina is updated in real-time.

🔁 Update Method (Runs Every Fixed Frame)

This is the heart of the stamina system. It decides whether to:

- **Drain stamina** (if the player is running and moving).
- **Regenerate stamina** (if the player is not running or not moving).
- **Disable running** (if stamina drops to 0).

- **Enable running** (if stamina goes above a certain threshold again).

It also updates the public status values (hasStamina, staminaPercentage, etc.).

🏃 StartRunning

Tries to set the player speed to "fast" if stamina is available.

If stamina is not used at all, it just sets the speed to fast.

🧍 StopRunning

Always sets the player speed back to normal and flags that the player is no longer running.

🧹 Dispose

Stops updating the controller every frame and clears the reference. Useful when removing or disabling the player.

🔌 SubscribeUpdate

Makes sure the Update method is called every physics frame. Only activates once to avoid multiple registrations.

🕹️ Usage Scenario

A common usage might look like this:

1. A character is walking by default at 3 units/sec.
2. When the player holds the "Run" button and stamina is available, the character moves at 6 units/sec.
3. Stamina drains while running.
4. When stamina hits zero, the speed goes back to walking.
5. If the player stops moving or running, stamina slowly regenerates.
6. Once stamina is above 25%, the player can run again.

✅ Why This Is Useful

- Adds depth to movement by limiting how long the player can sprint.
- Supports UI elements like a stamina bar.

- Flexible: can be reused on different characters by injecting different speed/stamina functions.
- Easy to disable the stamina system for simpler movement setups.

Player Utils

Purpose of the Script

This script is a **utility tool** for player movement and collision-related calculations, meant to support other systems (like movement controllers). Currently, it contains one **static helper function** that rotates a 2D direction vector by a specified number of degrees.

It's part of a **namespace** called PlayerController.Utils, suggesting it's designed to be reused by other gameplay components.

Function: ConvertRotation

What it Does

This method **rotates a 2D vector** (representing direction, like player input) by a given number of degrees **clockwise**. It returns a new vector with the rotation applied.

Input Parameters

Name	Description
direction	A 2D vector that defines the original direction (X, Y).
rotation	An angle in degrees to rotate the vector clockwise.

Output

The method returns a **new 2D vector** that points in the same direction as the original vector, but rotated by the given angle.

How It Works (Internally)

1. **Convert degrees to radians**
   Rotation calculations in mathematics use radians, so the angle is first converted from degrees.
2. **Compute cosine and sine**
   These values are used in a rotation matrix to rotate a vector in 2D space.
3. **Apply 2D rotation matrix**
   The function uses this formula to rotate the vector:
4. x' = x * cos(θ) - y * sin(θ)

5. y' = x * sin(θ) + y * cos(θ)

This creates a rotated version of the input vector.

6. **Return result**
   A new vector is created and returned with the rotated X and Y components.

🕹️ Example of Usage (Conceptual)

Let's say you have a movement system that gets input like:

direction = (0, 1) // Up

rotation = 90   // Rotate clockwise

Calling the method with these values would return:

rotated = (1, 0) // Now facing right

This is helpful if, for example:

- Your camera is rotated and you want player input to match it.
- You're applying directional forces but need to offset them by an angle.
- You want to rotate a visual indicator or aiming reticle based on analog stick direction.

📦 Summary of Usage

- Simple, reusable method for rotating 2D vectors.
- Useful in input, aiming, and directional movement.
- Commonly used in player movement scripts to align motion with camera or world orientation.

Physics Update Broadcaster

🧩 Purpose of the Script

This script creates a **global update broadcaster**. It serves two main purposes:

1. **Broadcasts the engine's update cycles** (normal update, physics update, and late update) through static events that other systems can subscribe to.
2. **Manages and visually displays** custom collision sensors (BoxCollisionSensor) as gizmos, depending on visibility settings.

⚙️ Core Components and Logic

🔁 Lifecycle Events

Three update cycles are broadcasted as global signals:

- OnUpdate: Triggered once per frame.
- OnFixedUpdate: Triggered every physics frame (used for physics-based systems).
- OnLateUpdate: Triggered after normal updates each frame.

These signals allow any other part of the game to **listen** and act during those cycles without needing to be attached to a game object.

🎯 Singleton Behavior

A variable stores the **singleton instance** of the class. During initialization:

- If another instance already exists, the new one destroys itself.
- If not, the current one becomes the **persistent global instance**, surviving scene changes.

This guarantees there's always **one global event broadcaster**.

📦 Registered Sensors List

A static list holds references to all the **collision sensors** (BoxCollisionSensor instances) that want to show visual debugging gizmos.

Sensors can be:

- **Added** using the method AddSensor(...)
- **Removed** using RemoveSensor(...)

This allows modular systems to register or unregister their visual debug tools dynamically.

🧰 Gizmo Drawing

The broadcaster will attempt to **draw a visual wireframe box** in the editor for each registered sensor, using these rules:

1. Each sensor provides:
    - Position.
    - Rotation.
    - Size.
    - Whether it should be visible always, or only when selected.
    - Color of the visual box.
2. The method OnDrawGizmos() is automatically called by the engine during scene rendering.
3. The broadcaster loops through all registered sensors and:
    - Draws the box if the sensor says it should **always be shown**.

- If in the editor, also draws it when the sensor is selected and configured to be shown only when selected.

The box is drawn as a **wireframe cube**, with its position, rotation, and size based on sensor values.

🧪 Auto Initialization

If the broadcaster doesn't exist when the game starts, it **automatically creates itself** and sets itself to persist across scene loads.

This is handled by a method marked to run during engine startup. It:

- Creates an empty game object.
- Attaches the broadcaster to it.
- Ensures it is never destroyed between scene loads.

🧠 Summary of Variables and Their Roles

Variable	Purpose
Instance	Stores the global broadcaster. Ensures only one exists.
OnUpdate / OnFixedUpdate / OnLateUpdate	Broadcasts engine update cycles to any listener.
registeredSensors	List of sensors to visualize via debug boxes.

🧪 Usage Example (Conceptual)

Imagine you have a character controller or enemy AI script and want to:

- Do something **every physics frame** → Subscribe to OnFixedUpdate.
- Show a **collision box** in the scene editor → Register your sensor via AddSensor(...).

Or, if you have a slope detector that wants to visualize its detection area only when selected in the editor — just configure it to return that option and provide its size/position.

🧰 Typical Use Cases

- Broadcasting a unified update loop for character movement, physics-based logic, or animations.
- Debugging collision checks and field-of-view areas in the editor.
- Managing global utility behaviors like stamina systems, ground detection, and velocity control without requiring each system to be a MonoBehaviour.

Camera State Manager

🎯 Purpose of the Script

This script manages whether the player's **camera is active or inactive**, based on:

- **Player input** (like pressing a button).
- **Other objects being active** (like menus, inventory, or dialogs).
- **Automatic update checks** based on the game loop (every frame or every physics update).

It is especially useful for pausing player control or hiding the camera when UI elements are open.

⚙️ Main Features

1. Can **enable or disable** the camera using:
    - Manual calls.
    - Input actions (like a keypress).
    - Automatic checks every frame (or every fixed interval).
2. Can react to the **active state of GameObjects** (like menus or panels) and disable the camera when they appear.
3. Has **event triggers** that are called whenever the camera is turned on or off.

🔁 Modes of Operation

There are four operational modes controlled by an option:

- **ManualToggle**: The camera can only be enabled/disabled through external calls.
- **InputEvent**: The camera toggles on/off when a specific player input is detected (like pressing a button).
- **Update**: The script checks object states every frame to decide if the camera should be enabled or disabled.
- **FixedUpdate**: The script does the same check, but only at fixed time intervals (used mostly for physics systems).

📦 Explanation of Each Variable

Variable	Description
inputAsset	The input actions used to capture player inputs.
toggleActionPath	The path to the specific action (like pressing the pause/menu button) that will toggle the camera.
updateType	Defines how the script will check or change the camera state (manual, by input, every frame, etc.).
activationObjects	A list of objects in the scene that influence the camera. If any of these are active, the camera will be disabled.

debug	If active, logs messages when the camera is toggled. Useful for development and testing.
onEnable	Event that is triggered when the camera becomes active. Can be used to turn on scripts, effects, etc.
onDisable	Event that is triggered when the camera is turned off. Useful for pausing gameplay or locking controls.
isCameraActive	Stores whether the camera is currently turned on.
lastActivationState	Tracks whether any of the watched objects were active in the previous check.
lastUpdateTime	Tracks the last time the script evaluated object states.
activeDuration / inactiveDuration	Track how long objects have been active or inactive, to avoid camera flickering due to rapid changes.
toggleEvent	Internal system to handle input events for toggling the camera.

🔄 How It Works

1. Initialization

When the script starts:

- It checks if an input configuration is provided.
- If yes, and input-based toggling is enabled, it starts listening to a specific input (e.g., "Menu" button).
- The camera starts in the **active** state by default.

2. Input-based Toggling

If configured to use input events:

- The script waits for a specific input (like a key/button press).
- When received, it **toggles** the camera on/off.

3. Object-based Camera Disabling

If automatic checking is enabled:

- The script continuously checks if **any object** from the configured list is active.
- If so, after a short delay (0.1 seconds), it disables the camera.
- If **no objects are active** for that same duration, the camera is re-enabled.

This delay prevents flickering from fast menu opening/closing.

4. Manual Camera Control

If set to **manual mode**, external systems (like other scripts or buttons) must call the function to turn the camera on or off.

5. Camera Toggling Logic

Whenever the camera state changes:

- The mouse cursor is either locked (for gameplay) or unlocked (for UI).
- Visibility of the cursor is updated.
- Configured events (onEnable, onDisable) are triggered.
- A debug message is shown, if debugging is enabled.

💡 Practical Examples

Scenario	Result
The inventory menu is opened (object becomes active)	After 0.1s, the camera is disabled.
The player presses the pause button	The camera toggles on/off immediately (if in input event mode).
A script manually calls the toggle method	The camera state changes accordingly.
All UI panels are closed	After 0.1s, the camera is enabled again automatically.

🧪 Use Cases

- Pausing gameplay when UI is open.
- Preventing camera movement when in a dialog or cutscene.
- Managing visibility and control flow when switching between game and interface.

Footstep Sound Controller

🎯 Purpose

This script plays **footstep and landing sounds** that match the player's movement style (walking, running, crouching, etc.). It uses a flexible method that works with **any movement controller**, as long as the controller provides certain information about the player's movement state.

The goal is to **create realistic, varied, and immersive audio feedback** depending on how the player moves.

🧩 Key Components and Roles

📌 Player Controller Reference

- This is the object that provides real-time information about the player's state (for example, whether they are moving, running, crouching, or grounded).
- It must support a specific set of movement-related properties, which the script accesses using a technique that reads properties at runtime (reflection).

📌 Audio Source

- This is the component that plays the actual sound clips.
- It can also be connected to an audio mixer group for volume, pitch, and effects control.

📌 Audio Clips

There are several groups of audio clips:

- **Walking/Running clips**: Used when the player is standing and moving.
- **Crouching clips**: Used when the player is crouching and moving.
- **Landing clips**: Optional sounds triggered when the player lands on the ground.

Each group allows multiple sound clips so the system can **randomize** them, making the audio feel more natural.

🔁 Script Behavior and Workflow

1. Initialization

- When the game starts:
  - It checks that both the player controller and audio components are correctly assigned.
  - It prepares to access movement data from the player controller.
  - It configures the sound system (volume, pitch range, mixer group).
  - It ensures the first footstep can play immediately.

2. Fixed Update Loop

- Every physics update:
  - It checks whether the player **just landed** (i.e., was in the air and is now on the ground).
  - If so, and landing sounds are enabled, it plays a **random landing clip**.
  - Then it evaluates the player's **current movement style** (walking, crouch-running, etc.).
  - Based on this style, it updates how often a footstep sound should play (the footstep interval).

- If enough time has passed, and the player is grounded and moving, it plays a new **footstep sound**.

🧠 Movement State Detection

The script evaluates four main flags from the player:

- Is the player moving?
- Is the player running?
- Is the player crouching?
- Is the player on the ground?

Based on these values, the player is categorized into one of the following movement states:

- Walking
- Running
- Crouch Walking
- Crouch Running
- None (e.g., idle or in the air)

Each movement state has a specific time interval between sounds:

- **Running** has shorter intervals (faster steps).
- **Crouch Walking** has longer intervals (slower steps).
- **None** disables footstep sounds entirely.

🔊 Sound Playback Logic

When a footstep is triggered:

- The script selects a **random sound clip** from the appropriate group (based on movement state).
- The sound is played using a **random pitch** between a minimum and maximum range to avoid repetition.
- If crouching, a softer sound is chosen; if running, the sound plays more frequently.

If the player just landed:

- A special **landing sound** is played (if enabled).
- If landing sounds are off, a regular footstep sound is used instead.

🧪 Use Case Example

Scenario	What Happens
Player starts walking	Footstep sounds play every 0.6 seconds, chosen randomly from the walking list.
Player starts running	Footsteps play faster (every 0.3 seconds).

Player crouch-walks	Steps slow down to 1 second apart, and different clips are used.
Player lands on the ground	A random landing sound is played (if enabled).
Player stands still	No sound is played.
Player jumps	Sound is paused while in the air.

✅ Summary

This system is designed for **flexibility, realism, and immersion**. You can plug in any player movement system that provides standard movement properties, and the sound playback will adapt accordingly — with randomized clips, pitch variation, landing detection, and support for different movement speeds or types.

Look Input Handler

🎯 Purpose

This component manages **camera look input**, such as mouse movement or right stick direction. It listens to a **player input action** and stores a two-dimensional value representing the **direction and strength of the look input**. This value can then be used by other systems (like a camera controller) to rotate the camera.

The system supports **enabling and disabling input** at runtime and is compatible with Unity's **Input System package**.

🧩 Variables & Their Roles

🔷 Input Action Asset

This is the collection of all input actions in the project. It includes a variety of actions like move, jump, and look. The asset must be assigned in the inspector.

🔷 Look Action Path

This string tells the script **which specific action** in the input asset should be used to track camera look input — typically mouse delta or right stick.

🔷 Look Direction (Vector2)

This value holds the **current input direction** for camera look. For example:

- Moving the mouse right increases the X value.
- Moving it up increases the Y value.
- When input is released, this value resets to zero.

🔷 Input Event Binding

This is a system used to **bind the input events** (start, hold, release) to methods. It updates the look direction value while the input is active, and clears it when the input stops.

🔷 Is Playable

A boolean flag that **controls whether input should be processed or ignored**. If this is turned off, the look direction is frozen and reset to zero. Useful when the game is paused or a menu is open.

🌀 Behavior and Workflow

🔶 Initialization

When the component is created:

- It checks if the input asset is assigned. If not, a warning is shown in the console.
- It does not yet listen to input events — this happens at the next stage.

🔶 Start Phase (Runtime Setup)

Once the game starts:

- The system binds the specified input action (from the look path) to two main behaviors:
  - **While Held**: It stores the directional input (e.g., how fast the mouse or stick is moving).
  - **On Release**: It resets the stored direction back to zero.
- However, input is only processed if the system is marked as "playable" (meaning not paused or disabled).

🔶 Input Update Logic

The look direction is updated continuously as long as the player moves the input device (mouse or stick) **and** the system is active.
This direction can be read by camera logic to rotate the view accordingly.

🔶 Input Disabling

At any time during gameplay, the system can be told to:

- Stop accepting input.
- Reset the stored direction to zero.

This is done by changing the isPlayable flag through a method that external systems (like a pause menu) can call.

🔶 Cleanup

When the component is destroyed (e.g., the object is removed or the scene changes):

- All input bindings are removed, which prevents memory leaks or unexpected behavior.

🧪 Usage Example

Scenario	Result
Player moves the mouse right	The lookDirection X value increases.
Player stops moving the mouse	lookDirection resets to zero.
Game is paused	Input is ignored, and lookDirection is cleared.
Right stick is moved up	The Y value of lookDirection increases.
Script is removed from scene	All event listeners are safely unregistered.

✅ Summary

This component acts as a **bridge between Unity's Input System and camera rotation logic**. It tracks camera input from the player and provides a clean and flexible way to:

- Capture and store directional input.
- Reset or ignore input based on game state.
- Easily plug into other camera control systems.

Player Animation Controller 3D

🎯 Purpose

This script is responsible for managing **player animations** in a **3D side-view game**. It ensures only **one animation state** is active at a time based on the player's current movement status, such as walking, running, jumping, crouching, or crawling. It uses **boolean parameters** in Unity's Animator system to activate and deactivate animations.

🧩 Variables & Their Roles

🔷 Player Controller (MonoBehaviour)

A reference to the player logic script that provides movement state data. This must implement the necessary movement properties through a standard interface.

🔷 Animator (Animator)

A reference to the Animator component that plays the animations. This must be assigned for the system to function.

🔷 Animation State Parameters

These are the names of the boolean parameters inside the Animator that trigger specific animations. They include:

- Idle
- Walking
- Running
- Jumping
- Crouch Idle
- Crouch Walking
- Crouch Running

These booleans are used to enable or disable specific animation clips.

🔷 Current State (Debug View)

This is a read-only variable used to track the current animation state internally. It helps avoid unnecessary transitions and can be viewed for debugging in the Unity Inspector.

🔷 Animator Hashes (Dictionary)

Instead of working with strings at runtime, the script converts parameter names into hashed values to **improve performance** and avoid typos. Each animation state is mapped to its hash for quick lookup.

🔷 Player Movement State Interface

This adapter reads the player's current condition using reflection or a shared interface. It exposes properties like:

- IsMoving
- IsRunning
- IsGrounded
- IsCrouching

🔁 How the Script Works

1. Initialization (Awake Phase)

When the game starts:

- The script checks if the required references are assigned.
- It sets up a **bridge** between this script and the player movement logic through a shared interface.

- Each animation state name is converted into a hash and stored for fast access.
- The animation system is initialized to the default "Idle" state, and all others are turned off.

2. Runtime Behavior (Update Phase)

Every frame:

- The script asks the player controller what the current movement state is.
- It decides which animation should be active based on that data.
- If the new state is different from the currently playing one:
  - The old animation is turned **off** by setting its boolean to false.
  - The new animation is turned **on** by setting its boolean to true.
  - The internal state tracker is updated to remember the new state.

This ensures that **only one animation boolean is active at a time**, preventing conflicting animations.

🧠 Decision Logic (State Selection)

The method that determines which animation to play uses the following logic:

Condition	Animation State
Grounded, not crouching, not moving	Idle
Grounded, not crouching, walking	Walking
Grounded, not crouching, running	Running
Not grounded, not crouching	Jumping
Grounded, crouching, not moving	Crouch Idle
Grounded, crouching, walking	Crouch Walking
Grounded, crouching, running	Crouch Running

If no conditions match (which shouldn’t happen), it falls back to the current state.

🧪 Usage Scenario

In a side-view 3D platformer:

- When the player walks → The "Walking" animation boolean becomes true.
- When the player crouches without moving → The "Crouch Idle" animation plays.
- When the player jumps → The "Jumping" animation plays.
- Only one of these is ever active, ensuring visual consistency.

✅ Summary

This script is a lightweight, robust animation manager designed for 3D side-view characters. Its key strengths are:

- **Only one active animation** at a time.
- Clear separation of player state and animation control.
- Flexible mapping using Animator boolean parameters.
- Optimized using hashed parameters.
- Easily extendable with new states or transitions.

It is ideal for projects that need **precise control over animation transitions** in third-person or side-scrolling gameplay.

Stamina Monitor

🎯 Purpose

This script is designed to **display and update a stamina bar UI** for a player. It reads the player's current **stamina percentage** and reflects it visually using a **UI slider** and an optional **text label**. The bar changes **color dynamically** depending on how full or empty the stamina is.

🧩 Components & Variables Explained

🔷 Player Controller Reference

- A reference to the player script that holds stamina data.
- It must contain a **public stamina percentage value**, so the script can read it using reflection.

🔷 Max Stamina & Min for Run

- maxStamina: Used to normalize stamina percentage (e.g., 0–100).
- minStaminaForRun: Defines the threshold below which stamina is considered **low**, and the slider color changes to yellow.

🔷 Slider Reference

- staminaSlider: The UI element (bar) that visually shows how much stamina the player has left.
- sliderFillImage: Internally stores the part of the slider that changes color and fills up.

🔷 Slider Color Settings

- emptyStaminaColor: Red. Displayed when stamina is 0%.
- lowStaminaColor: Yellow. Displayed when stamina is low (but above 0).
- normalStaminaColor: Green. Displayed when stamina is above the low threshold.

🔷 Stamina Text

- staminaText: Optional label that shows stamina in numbers (e.g., “45.6%”).

🔷 Adapter & Interface

- IPlayerStaminaInfo: Interface that guarantees access to a stamina percentage value.
- PlayerStaminaAdapter: A reflection-based adapter that reads stamina percentage from any script that has a property called StaminaPercent.

🔁 How It Works (Step-by-Step)

1. Initialization (Start)

- Calculates the **low stamina threshold** in percentage, based on the minimum stamina required to run.
- Configures the slider to use values from 0 to 100.
- Extracts the fill image from the slider so it can later change its color.
- Creates a **stamina adapter** to access the player's stamina dynamically using reflection.

2. Runtime Updates (Update)

Each frame:

1. The script checks if the player stamina reference is valid.
2. Reads the current stamina percentage.
3. Updates the slider's fill level to match the stamina.
4. Changes the slider's **color**:
    - **Red** if stamina is zero.
    - **Yellow** if stamina is below the threshold (but not zero).
    - **Green** if stamina is healthy.
5. Updates the optional text label to show the stamina as a percentage (e.g., "82%" or "82.3%").

🧪 Usage in a Game

In a typical gameplay scenario:

- As the player **runs**, **jumps**, or **performs actions** that consume stamina, the stamina bar visually **shrinks**.
- When stamina is depleted, the slider turns **red**.
- If the stamina is **low**, but not empty, the slider becomes **yellow**.
- When stamina is recovered (e.g., the player stops running), the bar fills back and returns to **green**.
- Optionally, the numeric percentage is shown next to the bar.

💡 Adaptability

- The script works with **any player script** that has a public stamina percentage.
- You can use this script without modifying your player code, as long as you follow the **property naming convention** (StaminaPercent).

- Easily integrates with **Unity UI** and **TextMeshPro**.

✅ Summary

The **Stamina Monitor** is a flexible and UI-focused script that:

- Dynamically reflects stamina status using a slider and color transitions.
- Reads data safely and generically via reflection.
- Provides feedback to the player about their stamina in a clear, visual way.
- Is easily reusable across multiple player characters or systems.

Perfect for **stamina-based gameplay mechanics** like sprinting, dodging, or stamina-limited abilities.

Player Prefab Manager

🧩 Purpose

This script is a **Unity Editor utility** designed to make it easier for developers to **quickly create and set up player prefabs** (pre-built player GameObjects) in the scene via **menu options**. It's **editor-only** and is not used during actual gameplay.

🔧 What It Does

- Adds **custom menu items** inside Unity's "GameObject" menu.
- When a developer clicks on one of these items (e.g., “3D → Base Player”), it:
    1. **Searches** for the correct prefab in the project files.
    2. **Instantiates** it under the currently selected GameObject in the hierarchy.
    3. **Unpacks** the prefab so it’s editable.
    4. **Selects it automatically**.
    5. **Triggers rename mode** so you can instantly rename the object.

📦 Variables and Functionality

🔷 InstantiateAndSetupPrefab(prefabName, parentObject)

This is the **main utility method**. It takes:

- A **string with the prefab name** to be instantiated.
- An optional **parent GameObject** under which the prefab will be created.

It performs the following:

1. **Searches** the project for prefabs matching the name.
2. Logs an error if none are found.
3. Loads the prefab asset.
4. Instantiates it under the specified parent.

5. Calls the setup method to finalize the process.

🔷 CompletePrefabSetup(prefabName, instantiatedObject)

Called after the prefab is added to the scene. It:

- Registers the object with Unity's **Undo system**, so actions can be undone.
- **Unpacks** the prefab instance (so it's no longer linked to the original prefab asset).
- **Selects** the newly created object in the editor.
- Waits until the next frame to simulate an **F2 key press**, opening the rename field.

This delayed rename ensures Unity has selected the object before trying to rename it.

🔷 Menu Methods

Each of these methods corresponds to a **menu item** in the Unity Editor and calls the main instantiation logic:

▪ **CreateBasePlayer**

Creates a generic **3D base player**.

▪ **CreateBasePlayerSideView**

Creates a **3D player prefab** designed for **side-scrolling gameplay**.

▪ **CreateBasePlayerFirstPerson**

Creates a **first-person player** prefab.

▪ **CreateBasePlayerThirdPerson**

Creates a **third-person player** prefab.

▪ **CreateBasePlayerThirdPersonCameraController**

Creates the **camera rig** used for third-person control.

Each of these menu items appears under:

GameObject → Player Controller → 3D → ...

🎯 Usage in Editor

When working in Unity Editor:

1. Open the **"GameObject" menu**.

2. Navigate to **"Player Controller → 3D"**.
3. Choose a prefab type (e.g., "Third Person").
4. The prefab is added directly into the current scene:
    - Under the currently selected object (or root if none selected).
    - Automatically unpacked for editing.
    - Automatically selected.
    - Renaming prompt opens.

This drastically **improves workflow**, especially for prototyping or level design.

🔐 Editor-Only Notes

- This script runs **only in the Unity Editor**.
- It uses UnityEditor-specific features like:
    - AssetDatabase: To find prefab files.
    - MenuItem: To create menu entries.
    - PrefabUtility: To instantiate and unpack prefabs.
    - Undo: To enable undo for creation.
    - Selection: To auto-select the new object.
    - EditorApplication.delayCall: To delay actions to the next frame.
    - EditorWindow.SendEvent: To simulate keypress (F2).

This means it is **not included in builds** and has **no effect at runtime**.

✅ Summary

This utility automates the process of placing player prefabs in your scene. It ensures:

- Correct prefabs are found and loaded.
- Prefabs are editable right after creation.
- User gets visual feedback with automatic selection.
- Developer can rename the object immediately.

It's a simple but powerful tool that speeds up development and keeps your scene organized.

Scene Dropdown Manager Editor

🧭 Purpose of the Script

This script creates a **dropdown menu in the Unity Editor UI** that allows developers to:

- View a list of scene files stored in a **ScriptableObject**.
- Select any of those scenes from a **dropdown list**.
- Load the selected scene into the editor or play mode **without using Build Settings**.

It is meant to be used **only within the Unity Editor** for debugging or navigation purposes.

🎮 How It Works

🔶 General Workflow:

1. On start, it reads the list of scenes from a ScriptableObject.
2. Displays the scene names in a dropdown menu (TextMeshPro format).
3. When the developer clicks the **Load Scene** button, the selected scene is loaded.

🧩 Variable Explanations

🖼️ sceneDropdown

- UI dropdown component from TextMeshPro (TMP) that displays all scene options to the developer.

🖱️ loadSceneButton

- Standard UI button component.
- When clicked, triggers the scene loading process.

📦 sceneList

- Reference to a **ScriptableObject** that stores scene assets manually.
- Each item in this object is a scene file (SceneAsset), allowing for scene control without using Unity's Build Settings.

📄 scenePaths

- Internal list storing **full asset paths** to the scenes listed in sceneList.
- These paths are used when loading the selected scene.

⚙️ Method Breakdown

🔧 Start()

This method is automatically triggered when the scene loads.

What it does:

1. Validates that the sceneList is assigned and contains scenes.
2. Calls SetupDropdown to populate the dropdown menu with scene names.
3. Registers a callback on the button to trigger scene loading when clicked.

📂 SetupDropdown()

This method:

1. Clears any old dropdown options and internal path list.
2. Reads each SceneAsset from the sceneList.
3. Converts each scene into:
    - A **label** (scene name, without the .unity extension).
    - A **path** (used for loading).
4. Fills the dropdown with the scene names.
5. Detects the currently open scene and sets it as the **default selected value**.
6. Updates the UI to reflect the selection.

This ensures the dropdown always reflects the current working scene and available options.

🚪 OnLoadSceneButtonClicked()

This method is called when the user clicks the load scene button.

What it does:

1. Validates the dropdown selection.
2. Retrieves the full path to the selected scene.
3. Checks whether Unity is in **Play Mode** or **Edit Mode**.
    - If in **Play Mode**, it loads the scene asynchronously in runtime.
    - If in **Edit Mode**, it opens the scene in the editor.

This ensures compatibility regardless of editor state.

📂 SceneListEditor (Referenced Object)

Although not shown in this script, the SceneListEditor is assumed to be a **custom ScriptableObject** containing a list of SceneAsset references. It allows developers to:

- Drag and drop scenes manually into a list.
- Avoid relying on Build Settings for scene navigation.

🧪 Usage Example

1. Attach this script to a GameObject in your scene (typically inside a debug menu).
2. Assign:
    - A TextMeshPro Dropdown.
    - A UI Button.
    - A SceneListEditor asset filled with scenes.
3. Enter play mode or stay in edit mode.
4. Select a scene from the dropdown.
5. Click **Load Scene**.

The selected scene is now loaded either in the editor or runtime.

🚫 Editor-Only Features

This script is wrapped in editor-specific checks (#if UNITY_EDITOR). It uses:

- EditorSceneManager: To open or load scenes.
- AssetDatabase: To retrieve asset paths.
- EditorApplication.isPlaying: To detect editor state.

These functions are **not included in builds** and will only execute inside the Unity Editor.

✅ Summary

This script is a **scene management tool for Unity developers**. It:

- Provides a visual way to list and switch between scenes.
- Works without touching Build Settings.
- Is highly useful during debugging or level design.

It improves productivity by eliminating the need to manually search and open scenes in the project window.

Scene List Editor

🧭 Purpose of the Script

This script defines a **custom asset** for the Unity Editor. It allows developers to **manually organize and store a list of scene files** (.unity scenes) using a drag-and-drop interface. It is designed to support editor tools (like a scene selection dropdown) without relying on the Unity Build Settings.

🧩 What This Script Does

- Creates a **ScriptableObject** called SceneListEditor.
- Inside this object, developers can add multiple scene files from the project.
- These scene references are used in editor scripts, such as a dropdown for switching scenes during development.

This is an **editor-only utility**, meaning it won't be included in game builds.

📂 Usage Context

The SceneListEditor asset is used by tools like SceneDropdownManagerEditor, which:

- Reads this list.
- Displays the scenes in a dropdown.
- Allows easy switching between scenes during testing.

This makes it easier for level designers or developers to test different scenes **without modifying Build Settings**.

📦 Script Components Breakdown

🏷️ CreateAssetMenu

This attribute allows you to create a new asset from the **Unity Editor menu**.

- Menu Path: “Player Controller → Debug → Scene List (Editor)”
- File Name: “SceneList.asset”

When used, this generates a .asset file that holds a list of scene references.

🧩 The Class

The main object is a **ScriptableObject**:

- A type of Unity asset used to store data.
- It exists as a .asset file in the project folder.
- It does not require being attached to a GameObject.

This class is only compiled in the Unity Editor, thanks to editor-specific compilation guards.

📄 The scenes List

This is the core variable:

- It is a **list of scene references** (SceneAsset type).
- These scenes are added manually by dragging .unity files from the Project window into the list.
- It is marked as public, so it’s visible and editable in the Unity Inspector.
- Only available in the Editor; this variable won’t exist in runtime builds.

🧪 How to Use It

1. **Create the asset**
   In the Unity Editor, go to the top menu:
2. Assets → Create → Player Controller → Debug → Scene List (Editor)
3. **Configure the asset**

    - Rename it as desired (e.g., "MyScenesList").
    - Drag scene files (from the Project window) into the scenes list field.
4. **Assign the asset to a scene loader**
   Use the created asset with tools like SceneDropdownManagerEditor, which will read this list to offer scene switching UI in development tools.

🚫 Editor-Only Behavior

This script is wrapped in a conditional compilation directive so that:

- The entire script is **only compiled when in the Unity Editor**.
- It uses types like SceneAsset, which only exist in the Unity Editor.
- It is **never included in game builds**, so it has no performance impact or file size cost in the final game.

✅ Summary

This script is a **lightweight asset container** that stores scene references for editor tools. It:

- Allows easy manual management of scenes.
- Supports editor tools for debugging or development workflows.
- Avoids reliance on Build Settings for editor-based scene loading.

In short, it's a developer-friendly asset used to streamline scene workflows in the Unity Editor.

Read Only In Inspector Attribute

🧭 What the Script Does

This script creates a **custom Unity attribute** that marks certain variables as **read-only in the Unity Inspector**. When applied to a field, Unity will still display the value, but developers won't be able to edit it directly in the editor.

Usage:
Add the attribute **[ReadOnlyInInspector]** above a variable to make it appear greyed out in the Inspector but still visible.

🎯 Why Use This?

- To show **debug information**, runtime values, or system-generated states in the Inspector.
- Prevent accidental editing of values that should only be modified via code.
- Improves UI clarity for developers and designers.

Script Breakdown

Custom Attribute: ReadOnlyInInspectorAttribute

This is a **marker attribute**, meaning it doesn't contain logic by itself. It simply tells Unity's editor drawer system:

"Hey, this field has a special rule."

- It inherits from Unity's built-in attribute base.
- This attribute can be combined with **[SerializeField]** to make a private variable show up in the Inspector but be non-editable.

How Unity Uses It

Unity uses custom attributes with special **drawers** that define how they appear in the Inspector. That's where the second part of the script comes in.

Custom Drawer: ReadOnlyInInspectorDrawer

This is a Unity Editor-only class that defines how fields marked with the custom attribute should be rendered.

Method: OnGUI(...)

This method is called whenever Unity draws the field in the Inspector. It receives:

- A rectangle describing the drawing area.
- The variable being drawn.
- The label shown in the Inspector.

What it does:

1. **Disables the GUI** system so the field cannot be edited.
2. **Draws the field anyway**, so its value is still visible.
3. **Re-enables the GUI** to avoid affecting other fields in the Inspector.

This makes the field greyed out (read-only) in the Inspector.

How to Use It in a Project

To use this attribute in your Unity scripts:

1. **Apply it to a field:**

Example:

[SerializeField, ReadOnlyInInspector]

private float currentHealth;

2. **Result in the Inspector:**
    - The currentHealth value will be visible in the Inspector.
    - It will be greyed out and cannot be edited manually.
    - It can still be updated by code at runtime.

💡 Key Notes

- This only affects the **Inspector UI**. It **does not make the variable read-only in code**.
- This works in both **Edit Mode** and **Play Mode**.
- This script is compiled only in the **Unity Editor**, so it has no impact on builds.

✅ Summary

This script provides a clean, reusable way to show non-editable variables in Unity's Inspector:

Feature	Purpose
Custom Attribute	Marks fields to be shown as read-only in the Inspector.
Custom Property Drawer	Controls the visual rendering of these fields (disables input).
Main Benefit	Prevents accidental edits while still allowing debugging and visibility.

This is a small but powerful tool for improving editor UX during development and debugging.

Tags Dropdown Attribute

✅ Purpose

Used to **ensure a string field matches a valid Unity tag**, which improves usability and avoids runtime errors caused by typos.

How to use it in your code:
Add **[TagsDropdown]** above any string field that should represent a Unity tag.

📦 Structure Overview

The script has two main parts:

1. **Attribute Definition** – Marks which fields should use the custom behavior.
2. **Custom Drawer** – Controls how the marked field is displayed in the Unity Editor.

This only affects the Inspector. The field still works as a normal string in your game logic.

🔍 Part 1 – The Attribute (TagsDropdownAttribute)

- It is a **marker attribute** with no logic inside.
- It simply flags fields to be handled by a custom drawer.

Why? Unity needs a way to identify which fields require special handling. This attribute is that signal.

🖌️ Part 2 – The Drawer (TagsDropdownDrawer)

This class overrides how fields with [TagsDropdown] are drawn in the **Unity Inspector**.

🔽 OnGUI(...) – Draws the Dropdown

This method controls the field's appearance in the Inspector.

What it does:

1. **Validates the field type**
   If the field is not a string, it shows a message saying the attribute is only for strings.
2. **Retrieves all Unity tags**
   Uses Unity's internal API to get the list of tags defined in the **Tag Manager**.
3. **Prepares a dropdown list**
   - Adds a special entry ("Tag Missing" or "Tag Missing (YourValue)") if the current value is not a valid tag.
   - Appends all valid Unity tags to the dropdown list.
4. **Displays the dropdown**
   - Shows the current tag selected.
   - If the user selects a valid tag, the string value is updated.
   - If the tag is missing, it shows a warning box below the field.

⚠️ GetPropertyHeight(...) – Calculates Field Height

This method tells Unity how much vertical space the field needs in the Inspector.

- If the tag value is invalid, it adds space for the warning box below the dropdown.
- If everything is fine, it returns the standard height.

📦 Fields in the Script

Field/Variable	Purpose
sceneDropdown	(Not in this script – leftover from other scripts; not used here)
TagsDropdownAttribute	Marks the string field to be rendered as a tag dropdown.
TagsDropdownDrawer	Custom editor drawer that replaces a text field with a tag dropdown.

tags	List of all tags defined in the Unity project.
currentString	The current value of the string field.
missingTagText	Label shown when the string doesn't match any tag.
tagList	Final list of tags shown in the dropdown (including warning option).

🧪 Example Usage

If you declare a variable like this:

[SerializeField, TagsDropdown]

private string spawnTag;

In the Inspector:

- You'll see a dropdown with all tags defined in **Unity's Tag Manager**.
- The selected tag becomes the value of spawnTag.
- If the value is invalid (e.g., if set from a script to a non-existent tag), the dropdown shows a warning.

🧠 Key Advantages

- Prevents human error: no typos in tag names.
- Increases clarity when configuring components.
- Adds editor-time validation (warning for missing tags).
- Works with existing Unity tag system — no custom tag systems required.

🧩 Summary Table

Component	Purpose
[TagsDropdown]	Attribute that marks string fields to use a tag dropdown.
OnGUI(...)	Draws a tag dropdown and handles selection.
GetPropertyHeight(...)	Adjusts height to show warning if tag is invalid.
Usage	[SerializeField, TagsDropdown] private string myTag;

This tool enhances the development workflow in Unity by making tag selection intuitive, error-resistant, and visually informative.

Validate Reference Attribute
This script creates a **custom attribute and property drawer** for Unity's Inspector that **validates object reference fields** (like GameObjects, components, or ScriptableObjects). When a

required reference is missing, it **highlights the field** and shows a message, either as a **warning** or an **error**, depending on configuration.

✅ Purpose

Used to make it visually obvious in the Inspector when an important reference has not been assigned. It helps catch mistakes during setup in the Unity Editor.

Usage Examples:

[SerializeField, ValidateReference] private GameObject player;

[SerializeField, ValidateReference(false)] private AudioClip optionalSound;

🧩 Components Breakdown

🔷 ValidateReferenceAttribute – The Marker

This is a **decorator** placed on object reference fields to flag them for validation.

Fields:

- UseError
  A boolean that defines how severe the message should be:
  - true → Display an **error** (red highlight).
  - false → Display a **warning** (yellow highlight).

Constructor:

- Accepts a parameter useError (default true).
- This allows flexible use: [ValidateReference], [ValidateReference(true)], or [ValidateReference(false)].

🔷 ValidateReferenceDrawer – The Drawer

This is the custom Inspector logic that renders the decorated field in a special way.

OnGUI(...) – Handles how the field looks

1. **Checks if the field is a reference** and whether it's null.
2. **Changes background color** of the field:
   - Red for error.
   - Yellow for warning.
3. **Draws the field** normally, but with the colored background.
4. If the field is still empty:

- Tries to identify the expected type of the missing object (e.g., GameObject, Rigidbody, etc.).
- Draws a **help box below the field** with a message like:

Put an item of type 'GameObject' here!

- The box is styled according to UseError.

GetPropertyHeight(...) – Calculates vertical space

- Returns enough space to fit both the field **and** the help message (if the field is empty).
- If the field has a value, it returns the normal single-line height.

📦 Variables Overview

Name	Purpose
UseError	Whether to show missing reference as an error (red) or warning (yellow).
isEmpty	Whether the property reference is null.
typeName	The expected object type (used in the message).
helpBoxRect	Layout rectangle for drawing the help box.
messageType	Determines whether Unity shows a warning or error icon.

🧪 Example Use Case

[SerializeField, ValidateReference(false)]

private AudioSource musicSource;

➕ In the Unity Inspector:

- If musicSource is unassigned:
  - The field is highlighted in **yellow**.
  - A warning box appears below:
    Put an item of type 'AudioSource' here!
- If you set ValidateReference(true) instead:
  - Highlight becomes **red** and message appears as an **error**.

🧠 Why It's Useful

- Prevents runtime null-reference errors by **highlighting configuration mistakes early**.
- Makes the Unity Editor more user-friendly and self-documenting.
- Encourages disciplined use of serialized references during scene or prefab setup.

Summary Table

Component	Description
ValidateReferenceAttribute	Flags fields for required validation.
UseError	Defines if the issue is treated as error or warning.
ValidateReferenceDrawer	Custom Inspector renderer for the attribute.
OnGUI()	Draws the field with a warning or error indicator if it's unassigned.
GetPropertyHeight()	Adds vertical space for the help box when needed.

This tool improves **editor-time validation**, provides **visual cues**, and contributes to better **scene/prefab safety** by catching missing references before the game is run.

Third Person Camera Controller

This script defines a **third-person camera system** for Unity. It includes **smooth rotation**, **zooming**, and **collision avoidance**. The system is designed to work with Unity's **Input System** and is highly configurable via the Inspector.

It attaches to a camera rig in the scene and follows a target (usually the player), allowing rotation around the character, adjustable zoom, and automatic repositioning to avoid camera clipping into objects.

Purpose

To provide a **smooth and flexible camera controller** for third-person games with:

- Mouse/touch/joystick control for looking around.
- Scroll wheel zoom.
- Automatic camera distance adjustment to prevent clipping.
- Y-axis inversion and sensitivity options.

Usage Summary

- Attach this script to a camera rig object.
- Assign required references: player, camera pivots, input handlers, etc.
- Bind the scroll wheel input to a zoom action (e.g., "UI/ScrollWheel").
- The camera will follow the player and respond to look and zoom input.

Workflow Overview

1. **Initialization (Awake and Start):**

    - Checks all required references.
    - Binds zoom input action.
    - Clamps the initial zoom distance.
2. **On Every Frame (Update):**
    - Processes look input to rotate the camera around the player.
    - Checks for objects between player and camera.
    - Moves the camera closer if there's a collision (without jitter).
3. **Public Controls:**
    - Change camera sensitivity.
    - Toggle Y-axis inversion.
    - Enable or disable camera processing logic.

🧩 Variable & Method Explanation

🔷 Input Settings

- inputActions: Input asset that defines input bindings (e.g., look and zoom).
- zoomActionPath: The path to the zoom binding inside inputActions (e.g., scroll wheel).

🔷 References

- lookInputHandler: Component that provides direction for camera look input.
- playerTransform: The player's main transform.
- playerPivotTransform: The point the camera orbits around.
- mainCameraPivotTransform: Used to rotate the camera in space.
- collisionCameraPivotTransform: The transform that moves closer when an obstacle is detected.

🔷 Camera Settings

- cameraSensitivity: General multiplier for how fast the camera rotates.
- angleSensitivity: Fine-tuning per-axis for yaw (left/right) and pitch (up/down).
- minPitch / maxPitch: Limits on how far up or down the camera can look.
- invertY: If true, looking up moves the camera down (classic invert).

🔷 Distance Settings (Zoom and Collision)

- minDistance / maxDistance: Min/max range of camera zoom.
- minCollisionDistance: How close the camera can get to the player when colliding.
- zoomSpeed: Controls how fast the scroll wheel zooms in/out.
- collisionDistanceSmoothSpeed: Speed at which the camera corrects distance when hitting a wall.
- targetCameraDistance: Desired camera distance; affected by scroll wheel and collision.
- currentCameraDistance: The actual camera distance (smoothed toward target distance).

🔷 Collision Settings

- collisionDetectionLayers: Which layers will be considered obstacles.
- collisionDetectionOffset: Padding so the camera doesn’t sit exactly on the hit surface.

🔁 Core Methods

🔶 Awake()

Checks if all required references are assigned. Displays warnings if not.

🔶 Start()

- Clamps initial camera distance.
- Binds the zoom input using a custom input handler (OnInputSystemEvent).
- Triggers an initial collision adjustment so the camera starts in a valid position.

🔶 Update()

Main logic loop:

- Handles look input for camera rotation.
- Handles collision detection to adjust camera distance accordingly.

🔶 OnDrawGizmosSelected()

Draws debug gizmos in the scene view when the object is selected.

🌀 Camera Rotation

ProcessLookInput()

- Takes the player's look direction input.
- Applies sensitivity and optional Y-axis inversion.
- Updates internal yaw and pitch angles.
- Applies rotation to the camera pivot.

🛡️ Camera Collision

ProcessCameraCollision()

- Casts a ray from the player to the camera’s desired position.
- If something is in the way:
    - Adjusts the camera’s Z position to sit just in front of the hit object.
    - Smooths the movement using Lerp to avoid jumps.

- Updates the local Z offset of the collision camera pivot.

🔧 Public Methods

Method	Purpose
SetCameraSensitivity(float)	Set rotation sensitivity (clamped between 0.05 and 1).
SetInvertYAxis(bool)	Enable or disable Y-axis inversion.
SetCameraActive(bool)	Enable or disable camera processing.

📌 Summary

Feature	How it Works
Look Rotation	From input handler → rotates pivot around player.
Zoom	Scroll input changes targetCameraDistance.
Clipping Avoidance	Raycast from player to camera → adjusts distance.
Y Inversion	Toggle pitch direction with invertY.
Smoothing	Zoom and collision response is interpolated.
Input System	Uses Unity's new Input System via InputActionAsset.

This controller is ideal for third-person games that need **polished camera control**, **zooming**, and **collision handling**, making it suitable for RPGs, shooters, or adventure games in 3D.

Third Person Player Controller 3D

This script defines a complete third-person player controller system designed for Unity. It manages input, movement, physics interaction, jumping, running, crouching, stamina, collision detection (with sensors), and camera control in a modular, extensible way.

🌐 Overall Concept

The player controller is a runtime system that receives input from the player, interprets that input according to game rules (e.g., can the player jump?), and applies that behavior to the player object using Unity's physics and transform systems. It supports both manual and automatic control modes, and can be extended via modular sensors and gameplay controllers.

🔌 Input and Player References

- **Input Asset & Paths**: These are mappings from the Unity Input System that define what actions (like jump or move) correspond to which keys or buttons.
- **Player Transform**: The main object in the scene that represents the player's position and orientation.

- **Camera Pivots**: References that manage where the camera is located in different states (standing or crouching).
- **Rigid Body and Colliders**: The rigid body enables physics interactions, while separate colliders define the player's hitboxes in standing vs crouching states.

🕹️ Movement Settings

- **Walk/Run Speeds**: These control how fast the player moves in different states (normal, crouched, running).
- **Jump Force and Count**: This defines how powerful jumps are and how many jumps can be done in a row.
- **Coyote Time**: A time buffer that allows a jump shortly after walking off a ledge.

🔋 Stamina System

- **Max Stamina and Min for Run**: Stamina is required for running. The system will prevent running when stamina is too low.
- **Depletion and Recovery Rates**: Stamina decreases while running and regenerates when idle.

🧠 Player State Enums

- **Control Mode**: Determines if player behavior is driven automatically by internal logic or manually overridden.
- **Ability**: Current capability (e.g., jumping or crouching).
- **Movement Mode**: Whether running is allowed or restricted to walking.

📦 Sensors and Collisions

Three main sensors are used for detecting the environment:

- **Ground Sensor**: Checks if the player is touching the ground.
- **Obstacle Sensor**: Checks for walls or other objects in front.
- **Ceiling Sensor**: Detects overhead obstructions to prevent standing up from crouch.

The system also includes:

- **Slope Sensor**: Checks if the ground angle is walkable.
- **Ignored Tags**: These allow objects like triggers or ghost platforms to be ignored by the sensors.

🧩 Controllers

- **Jump Controller**: Applies jump force when allowed.
- **Move Controller**: Moves the character, blocking it if there's an obstacle.
- **Rotation Controller**: Rotates the player in the direction of movement.
- **Stamina Controller**: Calculates movement speed based on stamina and state.

🎮 Input Event System

Each control (jump, crouch, move, run) is bound to an input event that triggers a response:

- **Jump**: Applies vertical force if the player is grounded or within coyote time.
- **Move**: Sets directional input, applying it to movement and rotation.
- **Crouch**: Toggles crouching based on player ability and ceiling clearance.
- **Run**: Starts or stops stamina drain and increases movement speed accordingly.

🔄 Update Cycle Logic

Each frame, the following happens:

1. **Transform Caching**: Saves current position and rotation for sensor calculations.
2. **Sensor Position Updates**: Adapts obstacle and ceiling sensors to crouching state.
3. **Ground Check**: Uses sensors to determine if the player is on the ground.
4. **Ability Handling**: Resolves crouch/stand transitions based on control mode and ceiling detection.
5. **Speed and Collider Management**: Adjusts speed and enables appropriate collider.
6. **Camera Positioning**: Smoothly shifts the camera between crouched and standing positions.

🪛 Public Properties

These allow other scripts to:

- Check if the player is grounded, crouching, running, or moving.
- Query stamina percentage.
- Check if the player is in a playable state.

📦 Save and Load System

- **SavePlayerData**: Collects current player state (mode, ability, position, etc.) and serializes it to a JSON string.
- **LoadPlayerData**: Restores the player state from a previously saved JSON string.

🩹 Cleanup System

- On destruction, the controller disposes of:
  - All sensor and controller instances.
  - All input bindings and listeners.

✅ Usage Summary

This script is designed for:

- Any 3D game in Unity that requires a modular and extendable third-person player controller.
- Games needing reliable crouch, run, jump, stamina, and slope mechanics.
- Developers who want a clear separation between input handling, physics logic, and player state.

First Person Camera Controller

🎮 What This Script Does

This component controls a **first-person camera** in Unity. It:

- Rotates the player's **horizontal direction** (left/right, or "yaw").
- Rotates the camera's **vertical pitch** (up/down).
- Supports **inverting Y-axis** (a common user preference).
- Has **sensitivity control** and **angle clamping** to prevent unnatural rotation.

This is commonly attached to a camera holder object in a first-person controller prefab.

🖇️ Dependencies / References

These are the required objects and components:

- **Look Input Handler**
  Receives the 2D input (usually mouse delta or joystick direction) for looking around.
- **Player Transform**
  Represents the physical body of the player. It rotates horizontally (yaw), which turns the whole character.
- **Camera Pivot**
  A child object of the player that holds the camera. This rotates vertically (pitch), allowing you to look up/down without tilting the player body.

⚙️ Configuration Settings

Sensitivity Control

- **Camera Sensitivity**
  A multiplier applied globally to all input movement. A lower value slows down camera movement; a higher value makes it faster. Acceptable range: 0.05 to 1.
- **Angle Sensitivity (Vector2)**
  A per-axis multiplier:
  - X axis: Yaw (left/right look).
  - Y axis: Pitch (up/down look).
    It allows adjusting rotation influence separately for horizontal and vertical movement.

Pitch Clamping

- **Minimum Pitch**
  Prevents looking down too far (e.g., past your own body).
- **Maximum Pitch**
  Prevents looking up too far (e.g., into the back of your head).

Together, they ensure the camera doesn't rotate beyond natural limits.

Y-Axis Inversion

- **Invert Y (Boolean)**
  If enabled, looking up moves the camera down, and vice versa. This matches preferences of some players from flight sims or classic shooters.

🔄 **Runtime Variable**

- **Current Pitch**
  Internally tracks how far the camera has looked up or down. It's updated every frame based on input and used to rotate the camera pivot.

🧠 **Core Logic (Per Frame)**

This is what happens each frame:

1. The script checks if the input and required objects are available.
2. It receives **2D look input** (horizontal and vertical values).
3. That input is:
   - Multiplied by **camera sensitivity**.
   - Then split into yaw and pitch.
   - Yaw rotates the **player's body** left/right.
   - Pitch rotates the **camera pivot** up/down.
4. The **pitch is clamped** between min and max limits to prevent unnatural rotation.

5. Finally, the **camera pivot** is rotated locally to reflect the new pitch.

🛠️ Public Methods

These are callable from other scripts or UI:

SetSensitivity

- **What It Does**: Sets the overall camera movement sensitivity.
- **Why Use It**: Useful for settings menus, allowing players to adjust mouse sensitivity.
- **Input**: A number between 0.05 and 1.

SetInvertY

- **What It Does**: Toggles whether up/down input is inverted.
- **Why Use It**: Lets users choose how vertical control feels (classic vs modern FPS feel).
- **Input**: True (invert enabled) or False (invert disabled).

✅ How to Use It

1. Attach this component to your **player controller prefab** or a camera rig.
2. Assign the following in the inspector:
    - A LookInputHandler that provides direction from the mouse or joystick.
    - A playerTransform object (typically the GameObject with the Rigidbody or CharacterController).
    - A cameraPivot (an empty GameObject that rotates vertically and has the actual camera as a child).
3. Tune sensitivity and pitch clamp values to your liking.
4. Call SetInvertY() or SetSensitivity() from a menu or settings manager.

🧩 Summary of Behavior

Feature	What It Controls
lookInputHandler	Where the input comes from
playerTransform	Yaw rotation (turn left/right)
cameraPivot	Pitch rotation (look up/down)
cameraSensitivity	Global speed control for camera
angleSensitivity	Per-axis multipliers
minPitch/maxPitch	Limits for vertical look
invertY	Player preference toggle

First Person Player Controller 3D

This script defines a comprehensive 3D first-person player controller built for Unity. It's designed to be modular, scalable, and extendable—ideal for advanced gameplay systems.

📌 General Purpose

This controller manages all aspects of a first-person character: movement (walk/run), jumping (including double jumps and coyote time), crouching, stamina management, ground and obstacle detection, and camera positioning. It uses Unity's Input System and physics-based interactions via rigidbody and colliders.

🎮 Input System

- **Input Assets & Paths:** The script connects to the Unity Input System via action maps (e.g., "Player/Jump", "Player/Move"). These strings tell the script which actions to listen for.
- **Events:** Each input (move, jump, crouch, run) is wrapped in a listener system that triggers logic when actions are pressed, held, or released.

📦 GameObject References

- **Player Transform & Rigidbody:** Defines the player's position and enables physics-based motion.
- **Colliders:** Two colliders represent the player's body—one for standing and one for crouching.
- **Camera Pivot:** This is the transform that the camera follows. It moves smoothly between standing and crouching positions.

🛠️ Movement Settings

- **Walk/Run Speeds:** Controls how fast the player moves, including reduced speed for crouching.
- **Jump Settings:** Defines jump strength and the number of jumps allowed (e.g., double jump).
- **Coyote Time:** Allows jumping for a brief moment after leaving the ground—makes platforming feel responsive.

💨 Stamina System

- **Stamina Max/Min:** Running depletes stamina; when it's too low, the player can't sprint.
- **Depletion/Recovery Rates:** Controls how fast stamina drains and replenishes.

🚦 Player States & Enums

- **Control Mode:** Can be automatic or manual. In manual, crouch is disabled.
- **Abilities:** Flags that determine whether the player can crouch or jump.
- **Movement Mode:** Defines if the player is allowed to run or just walk.

🧠 Sensors & Collision Detection

The script uses modular box-shaped sensors to detect environmental elements:

- **Ground Sensor:** Checks if the player is standing on walkable terrain.
- **Obstacle Sensor:** Detects walls and objects in front of the player to prevent walking through them.
- **Ceiling Sensor:** Used during crouch transitions to prevent the player from standing if there's something above.
- **Slope Angle Checker:** Validates if the ground is walkable based on steepness.

🎛️ Runtime Variables

These store current game state:

- **Flags:** Such as isCrouching, isGrounded, isRunning, isPlayable.
- **Speed Calculation:** Determines if walk/run speed is used, depending on crouching and stamina.
- **Cached Transform Data:** Holds position and rotation to use across sensors and logic.

📹 Camera Management

- The camera smoothly transitions between two pivot positions: standing and crouching.
- Lerp (interpolation) is used to create a smooth movement for immersion.

🔁 Main Loop (Update Cycle)

Each frame, the script:

1. Caches the current transform.
2. Updates the position of sensors.
3. Checks if the player is grounded.
4. Handles crouch logic based on player state and mode.
5. Calculates movement speed based on whether the player is running or crouching.
6. Updates the camera position accordingly.

🧩 Input Binding Setup

- **Jump:** Triggers if the player has the "CanJump" ability and is grounded.
- **Movement:** Converts 2D input (from keyboard or gamepad) to a 3D direction and applies it using physics.
- **Crouch:** Toggles crouch on press, except in manual mode.
- **Run:** Starts or stops running, depleting or recovering stamina.

🧪 Debug Tools

There are toggle flags that enable visual gizmos in the Unity editor for:

- Ground detection box
- Obstacle detection
- Ceiling check
- Slope validation

These help visualize and debug sensor behavior.

🔒 Public Interface

- **Getters:** Expose player states like crouching, grounded, stamina %, etc.
- **Setters:**
  - Set control mode (auto/manual)
  - Set abilities (jump/crouch)
  - Enable or disable run
  - Toggle playability (freeze/unfreeze the player)
- **Save/Load:** Can serialize and deserialize player state (position, rotation, ability, etc.) into JSON.

🧹 Cleanup (On Destroy)

When the controller is destroyed, it:

- Disposes of all sensors and controllers.
- Unbinds all input events to prevent memory leaks or crashes.

✅ Usage

To use this controller in a Unity project:

1. Attach it to a GameObject.

2. Assign all required references via the inspector (rigidbody, colliders, camera pivot, materials).
3. Plug in an InputActionAsset with properly mapped actions.
4. Customize speeds, jump strength, sensors, and materials as needed.

The system is highly customizable and extensible, making it ideal for advanced first-person game mechanics.

Side View Player Controller 3D

Your script implements a **modular 3D side-view player controller** in Unity, structured for extensibility and gameplay flexibility.

🔷 Overview

This controller handles everything from **movement, jumping, crouching, running**, and **rotation**, to **slope detection**, **stamina**, and **collision detection**. It uses a **sensor system** (box-based detection) and **input system event handlers**, making it modular and adaptable to various gameplay needs.

🔷 Input Settings

These variables link the controller to Unity's **Input System**:

- inputActions: The input asset that holds all player action maps.
- moveActionPath, runActionPath, jumpActionPath, crouchActionPath: Action paths used to bind specific input behaviors (e.g., WASD or gamepad stick).

🔷 References

These link the controller to necessary components:

- playerTransform: The player’s transform.
- playerRigidbody: The physics body responsible for movement.
- standingCollider and crouchingCollider: Used for toggling the hitbox when the player is crouching or standing.

🔷 Movement Settings

Defines how the player moves:

- walkSpeed and runSpeed: Speeds for walking and running.
- crouchWalkSpeed and crouchRunSpeed: Slower speeds while crouching.
- rotationSmoothSpeed: Controls how smoothly the player rotates.
- jumpForce, maxJumpCount, coyoteTimeDuration: Jump mechanics.

- globalInputRotationOffset: Applies rotation offset to movement input.

🔷 Stamina Settings

Defines stamina usage:

- maxStamina: Maximum stamina value.
- minStaminaForRun: Minimum stamina required to initiate running.
- staminaDepletionRate: Rate at which stamina drains when running.
- staminaRecoveryRate: Rate of stamina recovery when not running.

🔷 Player State and Collision Layers

- controlMode: Whether the player is controlled automatically or manually.
- currentAbility: The player's current ability (jumping, crouching).
- movementMode: Whether the player is allowed to run or only walk.
- groundLayers, obstacleLayers, ceilingLayers: Layers used for collision checks.

🔷 Sensors

These are 3D box-based sensors used to detect collisions:

- **Ground Sensor**: Checks if the player is on the ground.
- **Obstacle Sensor**: Detects objects in front of the player.
- **Ceiling Sensor**: Used to prevent standing up under low ceilings.
- **Slope Angle Sensor**: Checks if the player is on a valid walking surface.

Each sensor has configurable size, position offset, and ignored tags.

🔷 Physics Materials

- highFrictionMaterial, lowFrictionMaterial: Materials used to influence movement on slopes.

🔷 Runtime Fields

These are variables used during gameplay:

- Sensors and controllers: Objects handling jumping, movement, stamina, and rotation.
- Input event handlers: React to inputs like jumping, running, moving.
- State flags: Track if the player is grounded, running, crouching, playable.

- Cached positions and sizes: Used by sensors to detect changes efficiently.

🔷 Public Properties

- IsGrounded, IsCrouching, IsMoving, IsRunning: Runtime state accessors.
- StaminaPercent: Normalized stamina [0–100%].
- IsPlayable: Whether the player can be controlled right now.

🔷 Enums

Used to categorize control and player state:

- PlayerControlMode: Manual vs. Automatic.
- PlayerAbility: Abilities like jumping or crouching.
- PlayerMovement: Walking or running modes.

🔷 Lifecycle Methods

- Awake: Validates that all necessary components are assigned.
- Start: Initializes sensors, controllers, and input handlers.
- Update: Updates position caches, sensors, grounded state, ability logic, and movement speed.

🔷 Setup Logic

Sensors

- Each sensor is configured with position/size from the player's current transform.
- Sensors are color-coded for debugging: red (ground), yellow (obstacle), blue (ceiling).

Controllers

- JumpController: Handles single/double jumping with coyote time.
- MoveController: Handles forward movement with obstacle detection.
- RotationController: Smoothly rotates the player to face input direction.
- StaminaController: Controls speed, stamina drain/recovery, and run logic.

Input Events

- **Move**: Moves and rotates the player.
- **Jump**: Applies jump force if grounded or within coyote time.
- **Crouch**: Toggles crouching (only in Automatic mode).
- **Run**: Starts or stops stamina-based running.

🔷 Update Logic

CacheTransformData

- Stores player's current position and rotation for sensor use.

UpdateSensorPositions

- Updates box sensor positions based on crouch state and transform.

UpdateGroundedState

- Combines ground sensor and slope check to determine grounded status.

UpdateAbilityAndCrouchLogic

- Handles crouch toggling, ability switching, and ensures correct state.

UpdateSpeedAndColliders

- Switches between walk/run and crouch speeds.
- Enables/disables colliders based on crouch state.

🔷 Public API

These methods allow external control over the player:

- SetControlMode: Changes between manual and automatic modes.
- SetPlayerAbility: Grants or removes crouch/jump capabilities.
- SetRunEnabled: Enables/disables running globally.
- TogglePlayable: Disables/enables player input (e.g., during cutscenes).
- SavePlayerData: Saves key player data into JSON format.
- LoadPlayerData: Loads player data back from JSON, restoring abilities and position.

🔷 Cleanup

- DisposeAll: Cleans up all sensors, controllers, and unbinds inputs to avoid memory leaks.

🧪 Debugging

There are several debug flags to enable visualization of sensors and slope angles in the editor, which helps during development.

✅ Usage Summary

To use this system:

1. Assign the InputActionAsset and all required components (colliders, Rigidbody, materials).
2. Adjust movement, jump, and stamina parameters via inspector.
3. Make sure sensors are positioned and sized correctly.
4. Control the player using input bindings defined in the Input System.
5. You can extend it by adding new abilities, refining sensors, or creating new control modes.

Base Player 3D

Overview

This script is a **modular 3D player controller** for Unity. It manages player movement, jumping, running, crouching, stamina, input handling, and collision detection using configurable sensors and a modular runtime architecture.

Serialized Fields (Editable in Inspector)

These are the core configuration fields used to customize the player controller in the Unity Editor:

Input Settings

- inputActions: Defines the reference to the Input Action Asset used for player input.
- moveActionPath, runActionPath, jumpActionPath, crouchActionPath: Define action map paths used to identify each action from the Input Asset.

References

- playerTransform: The transform of the player.
- playerRigidbody: Rigidbody used for movement and physics.
- standingCollider, crouchingCollider: Two colliders used to switch between standing and crouching states.

Movement Settings

- walkSpeed, runSpeed: Control speed when walking or running.
- crouchWalkSpeed, crouchRunSpeed: Same as above, but used while crouching.
- rotationSmoothSpeed: Affects how smoothly the player rotates.
- jumpForce: Controls how strong the jump is.
- maxJumpCount: Maximum times the player can jump before landing.
- coyoteTimeDuration: Time window after falling where jump is still allowed.

- globalInputRotationOffset: Rotates input vector direction by a certain angle.

Stamina Settings

- maxStamina: Maximum stamina the player can have.
- minStaminaForRun: Minimum stamina required to start running.
- staminaDepletionRate: How fast stamina depletes while running.
- staminaRecoveryRate: How fast stamina regenerates when not running.

Player State

- controlMode: Determines if the player is controlled automatically or manually.
- currentAbility: Defines whether the player can jump or crouch.
- movementMode: Defines if the player can run or only walk.

Collision Layers

Used to identify the relevant objects for sensors:

- groundLayers, obstacleLayers, ceilingLayers: Layers checked for ground, obstacle, and ceiling collisions.

Sensor Settings

Define the box sizes and centers for the different sensors used in collision detection:

- groundSensorBoxSize
- obstacleSensorCenter, obstacleSensorSize
- crouchObstacleCenter, crouchObstacleSize
- ceilingSensorCenter, ceilingSensorSize
- Each group has a list of tags to ignore during detection (e.g., "Ignore Collision").

Physics Materials

- highFrictionMaterial, lowFrictionMaterial: Materials applied to simulate walking on different slopes.
- maxSlopeAngle: Maximum angle that is walkable.

Debug Flags

- Boolean flags to control whether gizmos are shown for debugging the sensors.

Runtime Fields (Internal Logic)

These fields are created at runtime to store game state, cache data, and manage systems.

Sensors

- groundSensor, obstacleSensor, ceilingSensor: Detect ground, forward collisions, and ceilings.

- slopeAngleSensors: Validate if the player stands on a walkable slope.

⚙ Controllers

- jumpController: Handles jumping logic, including double jump and coyote time.
- moveController: Applies movement to the Rigidbody.
- rotationController: Rotates the character smoothly in the movement direction.
- staminaController: Manages stamina and movement speed based on it.

🎮 Input Events

- jumpInputEvent, moveInputEvent, crouchInputEvent, runInputEvent: Bind the input system to game actions.

📦 Cached Values

- Player's position and rotation are cached to improve sensor calculations.
- currentMoveInput: The current movement direction.
- currentObstacleCenter, currentObstacleSize, currentCeilingCenter: Live values updated depending on crouch state.

🚦 State Flags

- currentSpeed: Current speed based on walking/running/crouching.
- isGrounded, isCrouching, isRunning, isPlayable: Boolean flags representing current state.
- canRun: Whether the player is currently allowed to run.

🌐 Public Properties

These are read-only values exposed for other scripts or UI systems:

- IsGrounded, IsCrouching, IsRunning, IsMoving, IsPlayable, StaminaPercent: Provide current state of the player.

🔄 Unity Event Methods

These are the Unity lifecycle methods:

- **Awake**: Validates inspector references.
- **Start**: Initializes components, sensors, and input bindings.
- **Update**: Called every frame. Handles caching, sensor updates, ability changes, and speed switching.
- **OnDestroy**: Cleans up all sensors, controllers, and input bindings.

🔧 Initialization Logic

Reference Validation

Ensures that all necessary references (InputAsset, Rigidbody, etc.) are assigned and warns in the console if they're missing.

Setup Sensors

Creates three box-shaped sensors and a slope angle validator:

- Ground sensor: Checks if the player is on the ground.
- Obstacle sensor: Detects walls and other objects in front.
- Ceiling sensor: Prevents the player from standing up while crouched.
- Slope sensors: Prevent player from walking on steep angles.

Setup Controllers

Initializes systems that manage jumping, movement, stamina, and rotation.

Setup Input Bindings

Binds input actions to in-game behaviors:

- Jump: Triggers jump controller if ability allows.
- Move: Applies directional movement.
- Crouch: Toggles crouching if automatic mode is active.
- Run: Starts/stops running and stamina depletion.

🔁 Update Logic (Every Frame)

- **Transform Caching**: Saves position and rotation for sensors.
- **Sensor Updates**: Changes sensor positions depending on crouch state.
- **Ground State Check**: Sets isGrounded based on collision and slope.
- **Ability Logic**: Resets crouch state in manual mode, disables jumping if no ability is active.
- **Speed/Collider Updates**: Switches movement speed and active collider based on crouch state and stamina.

🧩 Public API

Exposed methods for external access or UI buttons:

- **SetControlMode(index)**: Switches between manual or automatic control.
- **SetPlayerAbility(index)**: Allows enabling or disabling crouch/jump.
- **SetRunEnabled(bool)**: Toggles running mode.
- **TogglePlayable(bool)**: Disables or enables all player input.
- **SavePlayerData()**: Returns a JSON string with current player state.

- **LoadPlayerData(json)**: Restores player state from a saved JSON string.

🧹 Cleanup Logic

DisposeAll() is called when the object is destroyed:

- Disposes all sensors and controllers.
- Unbinds input actions.
- Frees up memory and avoids leaks or ghost behaviors.

🧪 Usage Summary

- Drag and drop this script onto a GameObject.
- Assign all required fields (InputAsset, Rigidbody, Colliders, Materials, etc.).
- Customize behavior via the Inspector or public API.
- Plug in external systems to respond to events or expose IsRunning, StaminaPercent, etc.